

SOLIDWORKS Document Manager API (.NET) — Examples (3/4):

Get_Configuration-Specific_Custom_Properties
Get_Persistent_IDS_Example_VBNET

SOLIDWORKS API Documentation (converted from CHM)

Get Configuration-Specific Custom Properties for Components (VB.NET)

This example shows how to get custom properties for assembly components.

```
'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
'    topic and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
'    in Microsoft Visual Studio.
' 3. Ensure the Solution Platform is of type x64.
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click:
'        install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
'    e. Click Add.
'    f. Click Close.
' 5. Add the Microsoft Scripting Runtime reference to the project.
' 6. Ensure that the model(s) to open exists and is(are) not readonly.
' 7. Substitute your_license_key with your SOLIDWORKS Document
'    Manager license key.
' 8. Open the Immediate window.
'
' Postconditions:
' 1. Gets document-level, component-level, and configuration-level custom properties for all assembly components.
' 2. Inspect the Immediate window.
'-----
```

Imports System.Diagnostics

Imports SolidWorks.Interop.swdocumentmgr

Module Module1

```
Sub Main()
    Dim swDocMgr As SwDMApplication
    Dim swCf As SwDMClassFactory
    swCf = CreateObject("SwDocumentMgr.SwDMClassFactory")
    Dim sKey As String
    sKey = "your_license_key"
    swDocMgr = swCf.GetApplication(sKey)

    Dim swDmSearchOpt As SwDMSearchOption
    swDmSearchOpt = swDocMgr.GetSearchOptionObject
    swDmSearchOpt.ClearAllSearchPaths()
    swDmSearchOpt.AddSearchPath("public_documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\advdrawings")

    Dim dOpenDocs As New Scripting.Dictionary

    Dim swDoc12 As SwDMDocument12
    Dim res As SwDmDocumentOpenError
    Dim dt As SwDmDocumentType
    dt = SwDmDocumentType.swDmDocumentAssembly
    Dim filename As String
    filename = "public_documents\SOLIDWORKS\SOLIDWORKS 2024\samples\tutorial\advdrawings\98food processor.SLDASM"
    swDoc12 = swDocMgr.GetDocument(filename, dt, False, res)

    If swDoc12 Is Nothing Or (res <> SwDmDocumentOpenError.swDmDocumentOpenErrorNone) Then
        Debug.Print("Error opening file..")
    End If
End Sub
```

```

Exit Sub
End If

dOpenDocs.Add(filename, swDoc12)
Dim activeConfig As SwDMConfiguration8
Dim configMgr As SwDMConfigurationMgr
configMgr = swDoc12.ConfigurationManager
Dim vConfigNames As Object
vConfigNames = configMgr.GetConfigurationNames
Dim j As Integer
Debug.Print("Assembly configurations: ")
For j = 0 To UBound(vConfigNames)
    Debug.Print(" " & vConfigNames(j))
Next j
Debug.Print("Get the active assembly configuration...")
Dim sActiveConfig As String
sActiveConfig = configMgr.GetActiveConfigurationName
activeConfig = configMgr.GetConfigurationByName(sActiveConfig)
If activeConfig Is Nothing Then
    Debug.Print("Error getting the active assembly configuration...")
    Return
End If
Debug.Print("The active assembly configuration is " & sActiveConfig)
Debug.Print("Get the components of the active assembly configuration...")

Dim vComponents As Object
vComponents = activeConfig.GetComponents
Dim swDmComponent As SwDMComponent12

Dim i As Integer
For i = 0 To UBound(vComponents)
    swDmComponent = vComponents(i)

    Debug.Print("")
    Debug.Print("Component name = " & swDmComponent.Name3 & "      Config = " & swDmComponent.ConfigurationName)

    Dim swDmCompDocCur As SwDMDocument
    Dim lErr As Integer
    Dim bIsInDict As Boolean
    bIsInDict = dOpenDocs.Exists(swDmComponent.Name)

    If Not bIsInDict Then
        swDmCompDocCur = swDmComponent.GetDocument2(False, swDmSearchOpt, lErr)
        If Not swDmCompDocCur Is Nothing Then
            dOpenDocs.Add(swDmComponent.Name, swDmCompDocCur)
        End If
    Else
        swDmCompDocCur = dOpenDocs(swDmComponent.Name)
    End If

    Dim vCustomPropNamesFromDoc As Object
    Dim lPropType As Integer
    Dim sPropVal As String
    Dim k As Integer

    If swDmCompDocCur IsNot Nothing Then

        Debug.Print("***** Document-level custom properties *****")

        vCustomPropNamesFromDoc = swDmCompDocCur.GetCustomPropertyNames()

```

```

If vCustomPropNamesFromDoc IsNot Nothing Then

    For k = 0 To UBound(vCustomPropNamesFromDoc)
        sPropVal = swDmCompDocCur.GetCustomProperty(vCustomPropNamesFromDoc(k), lPropType)
        Debug.Print(vCustomPropNamesFromDoc(k) & ": Value: " & sPropVal & ", Type: " & lPropType)
    Next k

    Debug.Print("***** Component-level custom properties *****")

    Dim m As Integer
    For m = 0 To UBound(vCustomPropNamesFromDoc)
        sPropVal = swDmComponent.GetCustomProperty(vCustomPropNamesFromDoc(m), lPropType)
        Debug.Print(vCustomPropNamesFromDoc(m) & ": Value: " & sPropVal & ", Type: " & lPropType)
    Next m

    Dim swDmCompDocCurConfigMgr As SwDMConfigurationMgr
    swDmCompDocCurConfigMgr = swDmCompDocCur.ConfigurationManager
    Dim swDmCompDocCurConfigMgr2 As SwDMConfigurationMgr2
    swDmCompDocCurConfigMgr2 = swDmCompDocCurConfigMgr

    Dim swDmCompConfigCur As SwDMConfiguration
    Dim swDmCompConfigCur14 As SwDMConfiguration14
    Dim swDmConfigErr As SwDMConfigurationError
    swDmCompConfigCur = swDmCompDocCurConfigMgr2.GetConfigurationByName2(swDmComponent.ConfigurationName, swDmConfigErr)
    swDmCompConfigCur14 = swDmCompConfigCur

    Dim lCustPropCount As Integer
    lCustPropCount = swDmCompConfigCur.GetCustomPropertyCount

    Dim vCustPropNames As Object
    vCustPropNames = swDmCompConfigCur.GetCustomPropertyNamees

    Debug.Print("***** Configuration-level custom properties *****")
    If lCustPropCount > 0 Then

        Dim r As Integer
        For r = 0 To UBound(vCustPropNames)
            sPropVal = swDmCompConfigCur.GetCustomProperty(vCustPropNames(r), lPropType)
            Debug.Print(vCustPropNames(r) & ": " & sPropVal & ", " & lPropType)
        Next r

    Else

        Debug.Print("None")

    End If

Else

    Debug.Print("None")

End If

Next

Dim s As Integer
Dim swDmDocCur As SwDMDocument
For s = 0 To dOpenDocs.Count - 1
    swDmDocCur = dOpenDocs.Items()(s)
    swDmDocCur.CloseDoc()
    swDmDocCur = Nothing
Next s
dOpenDocs.RemoveAll
dOpenDocs = Nothing

```

End Sub

End Module

Get Configuration Information Example (C#)

This example shows how to use the SOLIDWORKS Document Manager API to get configuration-related information for an external part document whose custom properties were transferred from the original part document.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
//    and ensure that the required DLLs have been registered.
// 2. Copy and paste this class into a C# console application in Microsoft
//    Visual Studio.
// 3. Add the Solidworks.Interop.swdocumentmgr.dll and Microsoft.CSharp references
//    to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click install_dir\api\redist\Solidworks.Interop.swdocumentmgr.dll.
//    e. Click Add.
//    f. Click Microsoft.CSharp.
//    g. Click Add.
//    h. Click OK.
// 4. In the code:
//    a. Substitute your_license_key with your SOLIDWORKS Document Manager license key.
//    b. Ensure that the file pointed to by sDocFileName exists.
// 5. Press F5 to run the program.
//
// Postconditions:
// 1. Examine the Immediate Window.
// 2. Verify that both fromparent+ prefaced and non-prefaced custom property
//    values are shown.
//-----

using System;
using System.Collections.Generic;
using System.Text;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;

class Program
{
    public static void ProcessDocCustomProperties(SwDMDocument19 swDoc)
    {
        string[] vCustPropNameArr = null;
        string sCustPropStr = null;
        SwDmCustomInfoType nPropType = 0;

        vCustPropNameArr = (string[])swDoc.GetCustomPropertyNames();
        if ((vCustPropNameArr == null)) return;

        Debug.Print(" Document Custom Properties:");

        foreach (string vCustPropName in vCustPropNameArr)
        {
            sCustPropStr = swDoc.GetCustomProperty(vCustPropName, out nPropType);
            Debug.Print(" Prefaced      = " + vCustPropName + " <" + nPropType + "> = " + sCustPropStr);

            sCustPropStr = swDoc.GetCustomProperty2(vCustPropName, out nPropType);
            Debug.Print(" Not prefaced = " + vCustPropName + " <" + nPropType + "> = " + sCustPropStr);

            Debug.Print("");
        }

        Debug.Print("");
    }

    public static void ProcessConfigCustomProperties(SwDMConfiguration14 swCfg)
    {
        string[] vCustPropNameArr = null;
        string sCustPropStr = null;
        SwDmCustomInfoType nPropType = 0;

        vCustPropNameArr = (string[])swCfg.GetCustomPropertyNames();

        if ((vCustPropNameArr == null)) return;
        Debug.Print(" Configuration Custom Properties:");

        foreach (string vCustPropName in vCustPropNameArr)
        {
            sCustPropStr = swCfg.GetCustomProperty(vCustPropName, out nPropType);
            Debug.Print(" Prefaced      = " + vCustPropName + " <" + nPropType + "> = " + sCustPropStr);
        }
    }
}
```

```

        sCustPropStr = swCfg.GetCustomProperty2(vCustPropName, out nPropType);
        Debug.Print(" Not prefaced = " + vCustPropName + " <" + nPropType + "> = " + sCustPropStr);

        Debug.Print("");
    }

    Debug.Print("");
}

static void Main(string[] args)
{
    const string sLicenseKey = "your_license_key";
    //Specify license key

    const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2021\\samples\\tutorial\\api\\ExternalReferencedPart.sldprt";
    //Specify document

    SwDMClassFactory swClassFact = default(SwDMClassFactory);
    SwDMApplication swDocMgr = default(SwDMApplication);
    SwDMDocument19 swDoc = default(SwDMDocument19);
    SwDMConfigurationMgr2 swCfgMgr = default(SwDMConfigurationMgr2);
    string[] vCfgNameArr = null;
    SwDMConfiguration17 swCfg = default(SwDMConfiguration17);
    SwDmDocumentType nDocType = 0;
    SwDmDocumentOpenError nRetVal = 0;

    SwDMConfigurationError results = 0;

    // Determine type of SOLIDWORKS file based on file extension

    if (sDocFileName.EndsWith(".sldprt")) {
        nDocType = SwDmDocumentType.swDmDocumentPart;
    }
    else if (sDocFileName.EndsWith(".sldasm")) {
        nDocType = SwDmDocumentType.swDmDocumentAssembly;
    }
    else if (sDocFileName.EndsWith(".slddrw")) {
        nDocType = SwDmDocumentType.swDmDocumentDrawing;
    }
    else {

        // Not a SOLIDWORKS file
        nDocType = SwDmDocumentType.swDmDocumentUnknown;

        // So cannot open
        return;
    }

    // Because drawing documents do not have configurations,
    // only continue running the project if the document
    // is a part or assembly document

    if ((nDocType != SwDmDocumentType.swDmDocumentDrawing)) {

        swClassFact = new SwDMClassFactory();
        swDocMgr = swClassFact.GetApplication(sLicenseKey);
        swDoc = (SwDMDocument19)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
        Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone == nRetVal);
        swCfgMgr = swDoc.ConfigurationManager;

        Debug.Print("File = " + swDoc.FullName);
        Debug.Print(" Active Configuration Name = " + swCfgMgr.GetActiveConfigurationName());
        Debug.Print("");
        Debug.Print(" Version = " + swDoc.GetVersion());
        Debug.Print(" Author = " + swDoc.Author);
        Debug.Print(" Comments = " + swDoc.Comments);
        Debug.Print(" Creation Date (string) = " + swDoc.CreationDate);
        Debug.Print(" Creation Date (numeric) = " + swDoc.CreationDate2);
        Debug.Print(" Keywords = " + swDoc.Keywords);
        Debug.Print(" Last Saved By = " + swDoc.LastSavedBy);
        Debug.Print(" Last Saved Date (string) = " + swDoc.LastSavedDate);
        Debug.Print(" Last Saved Date (numeric) = " + swDoc.LastSavedDate2);
        Debug.Print(" Subject = " + swDoc.Subject);
        Debug.Print(" Title = " + swDoc.Title);
        Debug.Print(" Last Update Timestamp = " + swDoc.GetLastUpdateStamp());
        Debug.Print(" Is Detached Drawing? " + swDoc.IsDetachedDrawing());
        Debug.Print("");

        vCfgNameArr = (string[])swCfgMgr.GetConfigurationNames2(out results);
        Debug.Print(" Number of configurations = " + wCfgMgr.GetConfigurationCount2(out results));

        foreach (string vCfgName in vCfgNameArr) {

            swCfg = (SwDMConfiguration14)swCfgMgr.GetConfigurationByName2(vCfgName, out results);

            Debug.Print(" " + vCfgName);
            Debug.Print(" Description = " + swCfg.Description);
            Debug.Print(" Parent Configuration Name = " + swCfg.GetParentConfigurationName());
            Debug.Print(" Last Update Timestamp = " + swCfg.GetLastUpdateStamp());

```

```
        Debug.Print(" 3DExperience configuration type as defined by  
swDm3DExperienceCfgType_e = " + swCfg.Get3DExperienceType());  
        Debug.Print("    3DExperience physical product represented by this  
configuration = " + swCfg.GetRepresentationParent());  
        Debug.Print("");  
        ProcessConfigCustomProperties(swCfg);  
    }  
    ProcessDocCustomProperties(swDoc);  
}  
}
```

Get Configuration Information Example (VB.NET)

This example shows how to use the SOLIDWORKS Document Manager API to get configuration-related information for an external part document whose custom properties were transferred from the original part document.

```

' -----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
'    and ensure that the required DLLs are registered.
' 2. Copy and paste this module into a VB.NET console application in Microsoft
'    Visual Studio.
' 3. Add the Solidworks.Interop.swdocumentmgr.dll reference to the project:
'    a. Right-click the solution in the Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click install_dir\api\redist\Solidworks.Interop.swdocumentmgr.dll.
'    e. Click Add.
'    f. Click OK.
' 4. In the code:
'    a. Substitute your_license_key with your SOLIDWORKS Document Manager license key.
'    b. Ensure that the file pointed to by sDocFileName exists.
' 5. Press F5 to run the program.
'
' Postconditions:
' 1. Examine the Immediate Window.
' 2. Verify that both fromparent+ prefaced and non-prefaced custom property
'    values are shown.
' -----

```

Imports SolidWorks.Interop.swdocumentmgr

Module Module1

```

Sub ProcessDocCustomProperties(ByVal swDoc As SwDMDocument19)

    Dim vCustPropNameArr As Object
    Dim vCustPropName As Object
    Dim sCustPropStr As String
    Dim sCustPropStrWOPrefix As String
    Dim nPropType As Integer

    vCustPropNameArr = swDoc.GetCustomPropertyNames : If IsNothing(vCustPropNameArr) Then Exit Sub
    Debug.Print(" Document Custom Properties:")

    For Each vCustPropName In vCustPropNameArr
        sCustPropStr = swDoc.GetCustomProperty(vCustPropName, nPropType)
        Debug.Print(" Prefaced = " & vCustPropName & " <" & nPropType & "> = " & sCustPropStr)

        sCustPropStrWOPrefix = swDoc.GetCustomProperty2(vCustPropName, nPropType)
        Debug.Print(" Not Prefaced = " & vCustPropName & " <" & nPropType & "> = " & sCustPropStrWOPrefix)

        Debug.Print("")
    Next

    Debug.Print("")
End Sub

Sub ProcessConfigCustomProperties(ByVal swCfg As SwDMConfiguration14)

    Dim vCustPropNameArr As Object
    Dim vCustPropName As Object
    Dim sCustPropStr As String
    Dim sCustPropStrWOPrefix As String
    Dim nPropType As Integer

    vCustPropNameArr = swCfg.GetCustomPropertyNames
    If IsNothing(vCustPropNameArr) Then Exit Sub
    Debug.Print(" Configuration Custom Properties:")

    For Each vCustPropName In vCustPropNameArr
        sCustPropStr = swCfg.GetCustomProperty(vCustPropName, nPropType)
        Debug.Print(" Prefaced = " & vCustPropName & " <" & nPropType & "> = " & sCustPropStr)

        sCustPropStrWOPrefix = swCfg.GetCustomProperty2(vCustPropName, nPropType)
        Debug.Print(" Not Prefaced = " & vCustPropName & " <" & nPropType & "> = " & sCustPropStrWOPrefix)

        Debug.Print("")
    Next

```

```

    Debug.Print("")

End Sub

Sub main()

    Const sLicenseKey As String = "your_license_key"
    Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2021\samples\tutorial\api\ExternalReferencedPart.sldprt"

    Dim swClassFact As SwDMClassFactory
    Dim swDocMgr As SwDMAApplication
    Dim swDoc As SwDMDocument19
    Dim swCfgMgr As SwDMConfigurationMgr2
    Dim vCfgNameArr As Object
    Dim vCfgName As Object
    Dim swCfg As SwDMConfiguration17
    Dim nDocType As Integer
    Dim nRetVal As Integer
    Dim results As Integer

    ' Determine type of SOLIDWORKS file based on file extension

    If InStr(LCase(sDocFileName), "sldprt") > 0 Then
        nDocType = SwDMDocumentType.swDmDocumentPart
    ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
        nDocType = SwDMDocumentType.swDmDocumentAssembly
    ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
        nDocType = SwDMDocumentType.swDmDocumentDrawing
    Else

        ' Not a SOLIDWORKS file
        nDocType = SwDMDocumentType.swDmDocumentUnknown

        ' So cannot open
        Exit Sub
    End If

    ' Because drawing documents do not have configurations,
    ' only continue running the project if the document
    ' is a part or assembly document

    If (nDocType <> SwDMDocumentType.swDmDocumentDrawing) Then

        swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
        swDocMgr = swClassFact.GetApplication(sLicenseKey)
        swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal) : Debug.Assert(SwDMDocumentOpenError.swDmDocumentOp
        swCfgMgr = swDoc.ConfigurationManager

        Debug.Print("File = " & swDoc.FullName)
        Debug.Print(" Active Configuration Name = " & swCfgMgr.GetActiveConfigurationName)
        Debug.Print("")
        Debug.Print(" Version = " & swDoc.GetVersion)
        Debug.Print(" Author = " & swDoc.Author)
        Debug.Print(" Comments = " & swDoc.Comments)
        Debug.Print(" Creation Date (string) = " & swDoc.CreationDate)
        Debug.Print(" Creation Date (numeric) = " & swDoc.CreationDate2)
        Debug.Print(" Keywords = " & swDoc.Keywords)
        Debug.Print(" Last Saved By = " & swDoc.LastSavedBy)
        Debug.Print(" Last Saved Date (string) = " & swDoc.LastSavedDate)
        Debug.Print(" Last Saved Date (numeric) = " & swDoc.LastSavedDate2)
        Debug.Print(" Subject = " & swDoc.Subject)
        Debug.Print(" Title = " & swDoc.Title)
        Debug.Print(" Last Update Timestamp = " & swDoc.GetLastUpdateStamp)
        Debug.Print(" Is Detached Drawing ? " & swDoc.IsDetachedDrawing)
        Debug.Print("")

        vCfgNameArr = swCfgMgr.GetConfigurationNames2(results)

        Debug.Print(" Number of configurations: " & swCfgMgr2.GetConfigurationCount2(results))

        For Each vCfgName In vCfgNameArr
            swCfg = swCfgMgr.GetConfigurationByName2(vCfgName, results)

            Debug.Print(" " & vCfgName)
            Debug.Print(" Description = " & swCfg.Description)
            Debug.Print(" Parent Configuration Name = " & swCfg.GetParentConfigurationName)
            Debug.Print(" Last Update Timestamp = " & swCfg.GetLastUpdateStamp)
            Debug.Print(" 3DExperience configuration type as defined by
swDm3DExperienceCfgType_e = " & swCfg.Get3DExperienceType)
            Debug.Print(" 3DExperience physical product represented by this
configuration = " & swCfg.GetRepresentationParent)
            Debug.Print("")

            ProcessConfigCustomProperties(swCfg)
        Next
    End If
End Sub

```

Next

ProcessDocCustomProperties(swDoc)

End If

End Sub

End Module

Get Current Name of Configuration of Suppressed Component (C#)

This example shows how to obtain the current name of the configuration of a suppressed component.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
//    and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
//    in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference
//    to the project:
//    a. Right-click the solution in Solution Explorer.
//    b. Click Add Reference.
//    c. Click Browse.
//    d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
//    e. Click Add.
//    f. Click OK.
// 4. Substitute your_license_key with your SOLIDWORKS
//    Document Manager license key.
// 5. Substitute assembly_with_suppressed_comps with the path to an assembly
//    that contains one or more suppressed components.
// 6. Open the Immediate window.
// Postconditions: Examine the Immediate window for configuration information.
//-----

using System;
using SolidWorks.Interop.swdocumentmgr;
using System.Diagnostics;
using System.IO;
using System.Collections;

namespace ConsoleApplication1
{
    class Program
    {
        static void Main()
        {
            //Specify your license key and path and filename of assembly document
            const string sLicenseKey = "your_license_key";
            const string sDocFileName = "assembly_with_suppressed_comps";

            SwDMClassFactory swClassFact = default(SwDMClassFactory);
            SwDMAApplication4 swDocMgr = default(SwDMAApplication4);
            SwDMDocument18 swDoc = default(SwDMDocument18);
            SwDMDocument18 swDoc2 = default(SwDMDocument18);
            SwDmDocumentOpenError nRetVal = 0;
            SwDMEExternalReferenceOption2 dmExtRefOption;
            SwDMSearchOption dmSearchOpt;
            int numExtRefs;

            swClassFact = new SwDMClassFactory();
            swDocMgr = (SwDMAApplication4)swClassFact.GetApplication(sLicenseKey);
            swDoc = (SwDMDocument18)swDocMgr.GetDocument(sDocFileName, GetDocType(sDocFileName), false, out nRetVal);
            Debug.Assert(SwDmDocumentOpenError.None == nRetVal);

            Debug.Print("File = " + swDoc.FullName);
            Debug.Print("");

            dmExtRefOption = swDocMgr.GetExternalReferenceOptionObject2();
            dmSearchOpt = swDocMgr.GetSearchOptionObject();
            dmExtRefOption.SearchOption = dmSearchOpt;
            dmExtRefOption.Configuration = "Default";
            dmExtRefOption.NeedSuppress = true;

            numExtRefs = swDoc.GetExternalFeatureReferences3(dmExtRefOption);
            Array arrExtRefs = (Array)(dmExtRefOption.ExternalReferences);

            SwDMConfiguration12 config;
            SwDMConfigurationMgr configMgr = swDoc.ConfigurationManager;
            config = (SwDMConfiguration12)configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName());
            object comps = config.GetComponents();
            Array arrComps = (Array)comps;

            foreach (SwDMComponent9 swComp in arrComps)
            {
                Debug.Print("Component name: " + swComp.Name2);
                Debug.Print("Component Configuration name: " + swComp.ConfigurationName);
                Debug.Print("Component Configuration ID: " + swComp.ConfigurationID);
                Debug.Print("Component Component ID: " + swComp.GetID());
                Debug.Print("");

                string ComponentPathName = swComp.PathName;
                //Check validity of the Component's path name
                //It might be out of date if the path changed after
                //the component was suppressed
                if (!File.Exists(ComponentPathName))
                    //If that file cannot be found, look for an external
            }
        }
    }
}
```

```

        //reference with the same name
        ComponentPathName = FindExtRefPath(swComp.PathName, arrExtRefs);

        swDoc2 = (SwDMDocument18)swDocMgr.GetDocument(ComponentPathName, GetDocType(ComponentPathName), false, out nRetVal);
        SwDMConfiguration12 config2;
        SwDMConfigurationMgr configMgr2 = swDoc2.ConfigurationManager;
        string[] confignames = (string[])configMgr2.GetConfigurationNames();
        ArrayList arrConfigNames = new ArrayList(confignames);

        // If the configuration name for the component doesn't match an
        // existing config name in the reference part, go find the updated
        // config name from the part
        if (!arrConfigNames.Contains(swComp.ConfigurationName))
        {
            foreach (string configName in arrConfigNames)
            {
                config2 = (SwDMConfiguration12)configMgr2.GetConfigurationByName(configName);
                if (config2.GetID() == swComp.ConfigurationID)
                    Debug.Print("Reference part configuration name: " + config2.Name2);
            }
        }

        swDoc2.CloseDoc();
    }
}

static string FindExtRefPath(string name, Array arrComps)
{
    string ExtrefPathName = name;
    string[] nameParts = name.Split('\\');
    string justName = nameParts.GetValue(nameParts.GetUpperBound(0)).ToString();

    int i = arrComps.GetLowerBound(0);
    Boolean found = false;
    while ((i <= arrComps.GetUpperBound(0)) && !found)
    {
        string extref = (arrComps.GetValue(i)).ToString();
        string[] extrefParts = extref.Split('\\');
        string justextrefName = extrefParts.GetValue(extrefParts.GetUpperBound(0)).ToString();
        if (justextrefName == justName)
        {
            found = true;
            ExtrefPathName = extref;
        }
        i++;
    }

    return ExtrefPathName;
}

static SwDmDocumentType GetDocType(string name)
{
    SwDmDocumentType nDocType = SwDmDocumentType.swDmDocumentUnknown;
    if (name.EndsWith("SLDPRT"))
    {
        nDocType = SwDmDocumentType.swDmDocumentPart;
    }
    else if (name.EndsWith("SLDASM"))
    {
        nDocType = SwDmDocumentType.swDmDocumentAssembly;
    }

    return nDocType;
}
}
}

```

Get Current Name of Configuration of Suppressed Component Example (VB.NET)

This example shows how to get the current name of the configuration of a suppressed component.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
'    and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
'    in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference
'    to the project:
'    a. Right-click the solution in Solution Explorer.
'    b. Click Add Reference.
'    c. Click Browse.
'    d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
'    e. Click Add.
'    f. Click OK.
' 4. Substitute your_license_key with your SOLIDWORKS
'    Document Manager license key.
' 5. Substitute assembly_with_suppressed_comps with the path to an assembly
'    that contains one or more suppressed components.
' 6. Open the Immediate window.
'
' Postconditions: Examine the Immediate window for configuration information.
'-----

```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports SolidWorks.Interop.swdocumentmgr
```

```
Imports System.IO
```

```
Module Module1
```

```
    'Specify your license key and path and filename of assembly document
```

```
    Const sLicenseKey As String = "your_license_key"
```

```
    Const sDocFileName As String = "assembly_with_suppressed_comps"
```

```
    Dim dmClassFact As SwDMClassFactory
```

```
    Dim dmDocMgr As SwDMApplication4
```

```
    Dim dmDoc As SwDMDocument18
```

```
    Dim dmDoc2 As SwDMDocument18
```

```
    Dim nRetVal As SwDmDocumentOpenError
```

```
    Dim dmSearchOpt As SwDMSearchOption
```

```
    Dim dmExtRefOption As SwDMExternalReferenceOption2
```

```
    Dim numExtRefs As Integer
```

```
    Sub Main()
```

```
        dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory.1")
```

```
        dmDocMgr = dmClassFact.GetApplication(sLicenseKey)
```

```
        dmDoc = dmDocMgr.GetDocument(sDocFileName, GetDocType(sDocFileName), True, nRetVal)
```

```
        Debug.Print("File = " + dmDoc.FullName)
```

```
        Debug.Print("")
```

```
        dmExtRefOption = dmDocMgr.GetExternalReferenceOptionObject2
```

```
        dmSearchOpt = dmDocMgr.GetSearchOptionObject
```

```
        dmSearchOpt.SearchFilters = (SwDmSearchFilters.SwDmSearchExternalReference + SwDmSearchFilters.SwDmSearchForAssembly + SwDmSearchFilters.SwDmSearchForConfiguration)
```

```
        dmExtRefOption.SearchOption = dmSearchOpt
```

```
        dmExtRefOption.Configuration = "Default"
```

```
        dmExtRefOption.NeedSuppress = True
```

```
        numExtRefs = dmDoc.GetExternalFeatureReferences3(dmExtRefOption)
```

```
        Dim arrExtRefs As Array
```

```
        arrExtRefs = dmExtRefOption.ExternalReferences
```

```
        Dim config As SwDMConfiguration12
```

```
        Dim configMgr As SwDMConfigurationMgr = dmDoc.ConfigurationManager
```

```
        config = configMgr.GetConfigurationByName(configMgr.GetActiveConfigurationName())
```

```
        Dim comps As Object = config.GetComponents()
```

```
        Dim arrComps As Array
```

```
        arrComps = comps
```

```
        For Each swComp As SwDMComponent9 In arrComps
```

```
            Debug.Print("Component name: " + swComp.Name2)
```

```
            Debug.Print("Component configuration name: " + swComp.ConfigurationName)
```

```
            Debug.Print("Component configuration ID: " + swComp.ConfigurationID.ToString)
```

```
            Debug.Print("Component ID: " + swComp.GetID.ToString)
```

```
            Debug.Print("")
```

```
            Dim ComponentPathName As String = swComp.PathName
```

```
            'Check validity of the component's path name
```

```
            'It might be out of date if the path changed after
```

```
            'the component was suppressed
```

```
            If Not File.Exists(ComponentPathName) Then
```

```
                'If that file cannot be found, look for an external
```

```
                'reference with the same name
```

```
                ComponentPathName = FindExtRefPath(swComp.PathName, arrExtRefs)
```

```

End If

dmDoc2 = dmDocMgr.GetDocument(ComponentPathName, GetDocType(ComponentPathName), False, nRetVal)
Dim config2 As SwDMConfiguration12
Dim configMgr2 As SwDMConfigurationMgr = dmDoc2.ConfigurationManager
Dim confignames As String()
confignames = configMgr2.GetConfigurationNames()
Dim arrConfigNames As New ArrayList(confignames)

' If the configuration name for the component doesn't match an
' existing config name in the reference part, go find the
' updated config name from the part
If Not arrConfigNames.Contains(swComp.ConfigurationName) Then

    For Each configName As String In arrConfigNames
        config2 = configMgr2.GetConfigurationByName(configName)
        If config2.GetID() = swComp.ConfigurationID Then
            Debug.Print("Reference part configuration name: " + config2.Name2)
        End If
    Next
End If

dmDoc2.CloseDoc()
Next
End Sub

Private Function FindExtRefPath(ByVal name As String, ByVal arrComps As Array) As String
    Dim ExtrefPathName As String = name
    Dim nameParts As String() = name.Split("\c")
    Dim justName As String = nameParts.GetValue(nameParts.GetUpperBound(0)).ToString()

    Dim i As Integer = arrComps.GetLowerBound(0)
    Dim found As [Boolean] = False
    While (i <= arrComps.GetUpperBound(0)) AndAlso Not found
        Dim extref As String = (arrComps.GetValue(i)).ToString()
        Dim extrefParts As String() = extref.Split("\c")
        Dim justextrefName As String = extrefParts.GetValue(extrefParts.GetUpperBound(0)).ToString()
        If justextrefName = justName Then
            found = True
            ExtrefPathName = extref
        End If
        i += 1
    End While

    Return ExtrefPathName
End Function

Private Function GetDocType(ByVal name As String) As SwDmDocumentType
    Dim nDocType As SwDmDocumentType = SwDmDocumentType.swDmDocumentUnknown
    If name.EndsWith("SLDPRT") Then
        nDocType = SwDmDocumentType.swDmDocumentPart
    ElseIf name.EndsWith("SLDASM") Then
        nDocType = SwDmDocumentType.swDmDocumentAssembly
    End If

    Return nDocType
End Function
End Module

```

Get DimXpert Block Tolerance Example (C#)

This example demonstrates how to use the SOLIDWORKS Document Manager API to acquire the following block tolerance values:

* ISO

* ASME

```
//-----  
// ISO:  
// 1. Open public_documents\samples\tutorial\dimxpert\shaft.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
//    (select the first instance of Toolbars on the View menu).  
// 3. Click Auto Dimension Scheme in the DimXpert toolbar.  
// 4. Click the green check mark to accept all settings.  
// 5. Close the part, saving it to another name.  
//    NOTE: Because this part is used in SOLIDWORKS tutorials,  
//          do not save changes to the original file name.  
// 6. Read the instructions to load and run Code Example CSharp  
//    with this part.  
// 7. Inspect the Immediate Window to see the ISO code for the part.  
//  
// ASME:  
// 1. Open:  
// public_documents\samples\tutorial\dimxpert\cover_with_geometric_tolerances.sldprt  
// 2. Close the part, saving it to another name.  
//    NOTE: Because this part is used in SOLIDWORKS tutorials,  
//          do not save changes to the original file name.  
// 3. Read the instructions to load and run Code Example CSharp  
//    with this part.  
// 4. Inspect the Immediate Window to see the ASME block tolerance  
//    values for the part.  
//-----
```

Get DimXpert Block Tolerance Example (VB.NET)

This example demonstrates how to use the SOLIDWORKS Document Manager API to acquire the following block tolerance values:

* ISO

* ASME

```
'-----  
' ISO:  
' 1. Open public_documents\samples\tutorial\dimxpert\shaft.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars  
'    (select the first instance of Toolbars on the View menu).  
' 3. Click Auto Dimension Scheme in the DimXpert toolbar.  
' 4. Click the green check mark to accept all settings.  
' 5. Close the part, saving it to another name.  
'    NOTE: Because this part is used in SOLIDWORKS tutorials,  
'          do not save changes to the original file name.  
' 6. Read the instructions to load and run Code Example VBNET  
'    with this part.  
' 7. Inspect the Immediate Window to see the ISO code for the part.  
'  
' ASME:  
' 1. Open:  
'    public_documents\samples\tutorial\dimxpert\cover_with_geometric_tolerances.sldprt  
' 2. Close the part, saving it to another name.  
'    NOTE: Because this part is used in SOLIDWORKS tutorials,  
'          do not save changes to the original file name.  
' 3. Read the instructions to load and run Code Example VBNET  
'    with this part.  
' 4. Inspect the Immediate Window to see the ASME block tolerance  
'    values for the part.  
'-----
```

Get DimXpert Chamfer Feature Example (C#)

This example builds a part that demonstrates usage of the SOLIDWORKS Document Manager API for a DimXpert chamfer feature.

```
//-----  
// 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
//    (select the first instance of Toolbars on the View menu).  
// 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
// 4. Ensure that the chamfer feature filter is selected.  
// 5. Click the green check mark to accept all settings.  
// 6. Observe the following DimXpert features on  
//    the DimXpertManager tab: Chamfer1.  
// 7. Save the part to another file name.  
//    NOTE: Because this part is used in a SOLIDWORKS online  
//          tutorial, do not save changes to the original file name.  
// 8. Close the part.  
// 9. Read the instructions for loading and running Code Example CSharp  
//    with this part.  
//-----
```

Get DimXpert Chamfer Feature Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert chamfer feature.

- ```
.....
' 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
' (select the first instance of Toolbars on the View menu).
' 3. Click Auto Dimension Scheme in the DimXpert toolbar.
' 4. Ensure that the chamfer feature filter is selected.
' 5. Click the green check mark to accept all settings.
' 6. Observe the following DimXpert features on the DimXpertManager tab:
' Chamfer1.
' 7. Save the part to another file name.
' NOTE: Because this part is used in a SOLIDWORKS online
' tutorial, do not save changes to the original file name.
' 8. Close the part.
' 9. Read the instructions for loading and running Code Example VBNET
' with this part.
.....
```

### Get DimXpert Cone Feature Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert entities:

- \* Cone feature
- \* Cone angle dimension tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\apilshaft.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars (select the
// first instance of Toolbars on the View menu).
// 3. Click Auto Dimension Scheme on the DimXpert toolbar.
// 4. Ensure that the Cone Feature Filter is selected.
// 5. Click the green check mark to accept the settings.
// 6. Observe the following DimXpert features on the
// DimXpertManager tab: Cone1 and ConeAngle1.
// 7. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes to the original
// file name.
// 8. Read how to load and run Code Example CSharp with this part.
//-----
```

Get DimXpert Cone Feature Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert entities:

- \* Cone feature
- \* Cone angle dimension tolerance

```
'-----
' 1. Open public_documents\samples\tutorial\apilshaft.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars (select the
' first instance of Toolbars on the View menu).
' 3. Click Auto Dimension Scheme in the DimXpert toolbar.
' 4. Ensure that the Cone Feature Filter is selected.
' 5. Click the green check mark to accept the settings.
' 6. Observe the following DimXpert features on the
' DimXpertManager tab: Cone1 and ConeAngle1
' 7. Close the part, saving it to another name.
' NOTE: Because this part is used in a SOLIDWORKS online
' tutorial, do not save any changes to the original
' file name.
' 8. Read how to load and run Code_Example_VBNET with this part.
'-----
```

### Get DimXpert Extrude Feature Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert extrude feature.

```
//-----
// 1. Open public_documents\samples\tutorial\api\plate_ads_plusminus.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
// (select the first instance of Toolbars on the View menu).
// 3. Create the Extrude Feature:
// a. Click Auto Dimension Scheme on the DimXpert toolbar.
// b. Ensure that all feature filters are selected.
// c. Click the green check mark to accept the settings.
// 4. Observe the following DimXpert features on the
// DimXpertManager tab: Hole Pattern1 and Pocket1.
// 5. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes when you close
// the original file name.
// 6. Read how to load and run Code Example CSharp.
//-----
```

### Get DimXpert Extrude Feature Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert extrude feature.

- ```
.....
* 1. Open public_documents\samples\tutorial\api\plate_ads_plusminus.sldprt.
* 2. Open the DimXpert toolbar from View > Toolbars
*   (select the first instance of Toolbars on the View menu).
* 3. Create the Extrude Feature:
*   a. Click Auto Dimension Scheme on the DimXpert toolbar.
*   b. Ensure that all feature filters are selected.
*   c. Click the green check mark to accept the settings.
* 4. Observe the following DimXpert features on the
*   DimXpertManager tab: Hole Pattern1 and Pocket1.
* 5. Close the part, saving it to another name.
*   NOTE: Because this part is used in a SOLIDWORKS online tutorial,
*   do not save any changes when you close the original file name.
* 6. Read how to load and run Code Example VBNET.
.....
```

Get More DimXpert Feature Examples (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Compound hole
- * Compound notch

```
//-----  
// 1. Open public_documents\samples\tutorial\apilcover_with_dimensions.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
// (select the first instance of Toolbars on the View menu).  
// 3. Observe the following DimXpert features on the DimXpertManager tab:  
// Simple Hole2 and Notch1.  
// 4. Close the part, saving it to another name.  
// NOTE: Because this part is used in a SOLIDWORKS online tutorial,  
// do not save any changes to the original file name.  
// 5. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get More DimXpert Feature Examples (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Compound hole
- * Compound notch

```
'-----  
' 1. Open public_documents\samples\tutorial\apilcover_with_dimensions.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars  
' (select the first instance of Toolbars on the View menu).  
' 3. Observe the following DimXpert features on the  
' DimXpertManager tab: Simple Hole2 and Notch1.  
' 4. Close the part, saving it to another name.  
' NOTE: Because this part is used in a SOLIDWORKS  
' online tutorial, do not save any changes to the original file name.  
' 5. Read how to load and run Code Example VBNET with this part.  
'-----
```

Get DimXpert Feature Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Cylinder
- * Fillet
- * Plane
- * Compound closed slot 3D
- * Pattern

```
//-----  
// 1. Open public_documents\samples\tutorial\apilcover_plate.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars.  
// 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
// 4. Ensure that all feature filters are selected.  
// 5. Click the green check mark to accept the settings.  
// 6. Observe the following DimXpert features on the DimXpertManager tab:  
//    Cylinder, Fillet, Plane, Slot, and Slot Pattern.  
// 7. Close the part, saving it to another name.  
//    NOTE: Because this part is used in a SOLIDWORKS online tutorial,  
//    do not save any changes to the original file name.  
// 8. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get DimXpert Feature Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Cylinder
- * Fillet
- * Plane
- * Compound closed slot 3D
- * Pattern

```
'-----'
' 1. Open public_documents\samples\tutorial\apicover_plate.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars.
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.
' 4. Ensure that all feature filters are selected.
' 5. Click the green check mark to accept the settings.
' 6. Observe the following DimXpert features on the DimXpertManager tab:
'    Cylinder, Fillet, Plane, Slot, and Slot Pattern.
' 7. Close the part, saving it to another name.
'    NOTE: Because this part is used in a SOLIDWORKS online tutorial,
'    do not save any changes to the original file name.
' 8. Read how to load and run Code Example VBNET with this part.
'-----'
```

Get DimXpert Intersect Features Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Intersect Point
- * Intersect Line
- * Intersect Circle
- * Intersect Plane

```
//-----
// 1. Open public_documents\samples\tutorialapi\face_plate_ads_geo.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
//    (select the first instance of Toolbars on the View menu).
// 3. Create a DimXpert Intersect Point Feature:
//    a. Click Location Dimension on the DimXpert toolbar.
//    b. Select the front face of the part.
//    c. Click Create Intersection Point on the DimXpert pop-up menu.
//    d. Select a cylinder whose axis intersects the first face at a point.
//    e. Click the green check mark in the pop-up menu to create the
//       intersect point.
//    f. Select a feature on the part to dimension against the intersect point.
//    g. Click outside the part to position the dimension in the view.
//    h. Observe the new DimXpert feature on the DimXpertManager tab:
//       Intersect Point1
// 4. Create a DimXpert Intersect Line Feature:
//    a. Click Location Dimension on the DimXpert toolbar.
//    b. Select the front face of the part.
//    c. Click Create Intersection Line on the DimXpert pop-up menu.
//    d. Select a perpendicular plane that would intersect the first plane
//       if extended (e.g., a top or side face of the part)
//    e. Click the green check mark in the pop-up menu to create the
//       intersect line.
//    f. Select a feature on the part to dimension against the intersect line.
//    g. Click outside the part to position the dimension in the view.
//    h. Observe the new DimXpert feature on the DimXpertManager tab:
//       Intersect Line1
// 5. Create a DimXpert Intersect Circle Feature:
//    a. Click Size Dimension on the DimXpert toolbar.
//    b. Select the front face of the part.
//    c. Click Intersect Circle on the DimXpert pop-up menu.
//    d. Select the opening conical surface of a flat head machine screw
//       (LPattern1 or LPattern3) in the part.
//    e. Click the green check mark in the pop-up menu to finish the dimension.
//    f. Click outside the part to position the size dimension in the view.
//    g. Observe the new DimXpert feature on the DimXpertManager tab:
//       Intersect Circle1
// 6. Create a DimXpert Intersect Plane Feature:
//    a. Click Location Dimension on the DimXpert toolbar.
//    b. Zoom in on a flat head machine screw (LPattern1 or LPattern3).
//    c. Select the opening conical surface of a flat head machine screw
//       in the part.
//    d. Click Intersect Plane on the DimXpert pop-up menu.
//    e. Select the inner cylindrical bore face of the flat head machine screw.
//    f. Click the green check mark in the pop-up menu to create the
//       intersect plane.
//    g. Select the top face of one of the extruded entities.
//    h. Click outside the part to position the location dimension in the view.
//    i. Observe the new DimXpert feature on the DimXpertManager tab:
//       Intersect Plane1
// 7. Close the part, saving it to another name.
//    NOTE: Because this part is used in a SOLIDWORKS online tutorial,
//         do not save any changes to the original file name.
// 8. Read how to load and run Code Example CSharp with this part.
//-----
```

Get DimXpert Intersect Features Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- * Intersect Point
- * Intersect Line
- * Intersect Circle
- * Intersect Plane

* 1. Open *public_documents\samples\tutorial\api\face_plate_ads_geo.sldprt*.

* 2. Open the DimXpert toolbar from **View > Toolbars** (select the first instance of Toolbars on the View menu).

* 3. Create a DimXpert Intersect Point Feature:

- * a. Click **Location Dimension** on the DimXpert toolbar.
- * b. Select the front face of the part.
- * c. Click **Create Intersection Point** in the DimXpert pop-up menu.
- * d. Select a cylinder whose axis intersects the first face at a point.
- * e. Click the green check mark in the pop-up menu to create the intersect point.
- * f. Select a feature on the part to dimension against the intersect point.
- * g. Click away from the part to position the dimension in the view.
- * h. Observe the new DimXpert feature on the DimXpertManager tab:
* **Intersect Point1**

* 4. Create a DimXpert Intersect Line Feature:

- * a. Click **Location Dimension** on the DimXpert toolbar.
- * b. Select the front face of the part.
- * c. Click **Create Intersection Line** on the DimXpert pop-up menu.
- * d. Select a perpendicular plane that would intersect the first plane if extended (e.g., a top or side face of the part)
- * e. Click the green check mark in the pop-up menu to create the intersect line.
- * f. Select a feature on the part to dimension against the intersect line.
- * g. Click away from the part to position the dimension in the view.
- * h. Observe the new DimXpert feature on the DimXpertManager tab:
* **Intersect Line1**

* 5. Create a DimXpert Intersect Circle Feature:

- * a. Click **Size Dimension** on the DimXpert toolbar.
- * b. Select the front face of the part.
- * c. Click **Intersect Circle** on the DimXpert pop-up menu.
- * d. Select the opening conical surface of a flat head machine screw (LPattern1 or LPattern3) in the part.
- * e. Click the green check mark in the pop-up menu to finish the dimension.
- * f. Click away from the part to position the size dimension in the view.
- * g. Observe the new DimXpert feature on the DimXpertManager tab:
* **Intersect Circle1**

* 6. Create a DimXpert Intersect Plane Feature:

- * a. Click **Location Dimension** on the DimXpert toolbar.
- * b. Zoom in on a flat head machine screw (LPattern1 or LPattern3).
- * c. Select the opening conical surface of a flat head machine screw in the part.
- * d. Click **Intersect Plane** on the DimXpert pop-up menu.
- * e. Select the inner cylindrical bore face of the flat head machine screw.
- * f. Click the green check mark in the pop-up menu to create the intersect plane.
- * g. Select the top face of one of the extruded entities.
- * h. Click away from the part to position the location dimension in the view.
- * i. Observe the new DimXpert feature on the DimXpertManager tab:
* **Intersect Plane1**

* 7. Close the part, saving it to another name.

* NOTE: Because this part is used in a SOLIDWORKS online tutorial, do not save any changes to the original file name.

* 8. Read how to load and run [Code Example VBNET](#) with this part.

Get DimXpert Sphere and Datum Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert entities:

- * Sphere feature
- * Datum annotation

```
//-----  
// 1. Open public_documents\samples\tutorial\floypress\ball valve\Ball.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars (select the first  
// instance of Toolbars on the View menu).  
// 3. Click Datum on the DimXpert toolbar.  
// 4. Click the ball of the part.  
// 5. Click outside the part to place the datum annotation.  
// 6. Click the green check mark to accept the new datum feature.  
// 7. Observe the following DimXpert feature on the DimXpertManager tab:  
// Sphere1.  
// 8. Close the part, saving it to another name.  
// NOTE: Because this part is used in a SOLIDWORKS online tutorial,  
// do not save any changes when you close the original file name.  
// 9. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get DimXpert Sphere and Datum Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert entities:

- * Sphere feature
- * Datum annotation

- ```
' -----
' 1. Open public_documents\samples\tutorial\floxpress\ball valve\Ball.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars (select the first
' instance of Toolbars on the View menu).
' 3. Click Datum on the DimXpert toolbar.
' 4. Click the ball of the part.
' 5. Click outside the part to place the datum annotation.
' 6. Click the green check mark to accept the new datum feature.
' 7. Observe the following DimXpert feature on the DimXpertManager
' tab: Sphere1.
' 8. Close the part, saving it to another name.
' NOTE: Because this part is used in a SOLIDWORKS online tutorial,
' do not save any changes when you close the original file name.
' 9. Read how to load and run Code Example VBNET with this part.
' -----
```

### Get DimXpert Surface Feature Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert surface feature.

```
//-----
// 1. Open:
// public_documents\samples\tutorial\moldedproductdesignadvanced\mousesplit.sldprt
// 2. Open the DimXpert toolbar from View > Toolbars
// (select the first instance of Toolbars on the View menu).
// 3. Create the DimXpert surface feature:
// a. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Profile of Surface from the Symbol dropdown
// list in the first row.
// ii. Click the top-right surface of the mouse.
// iii. Click outside the part to place the annotation.
// iv. Click OK to close the Geometric Tolerance
// Properties dialog.
// b. Observe SurfaceProfile1 under Profile Feature1 in the DimXpert tree.
// 4. Close the part, saving it to another name.
// NOTE: Because these parts are used in a SOLIDWORKS
// online tutorial, do not save any changes to the
// original file name.
// 5. Read how to load and run Code Example CSharp with this part.
//-----
```

### Get DimXpert Surface Feature Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert surface feature.

```
'-----
' 1. Open
' public_documents\samples\tutorial\moldedproductdesignadvanced\mouseplit.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
' (select the first instance of Toolbars on the View menu).
' 3. Create the DimXpert surface feature:
' a. Click Geometric Tolerance on the DimXpert toolbar.
' In the Geometric Tolerance Properties dialog:
' i. Select Profile of Surface from the Symbol dropdown
' list in the first row.
' ii. Click the top-right surface of the mouse.
' iii. Click outside the part to place the annotation.
' iv. Click OK to close the Geometric Tolerance Properties dialog.
' b. Observe SurfaceProfile1 under Profile Feature1 in the DimXpert tree.
' 4. Close the part, saving it to another name.
' NOTE: Because these parts are used in a SOLIDWORKS online tutorial,
' do not save any changes to the original file name.
' 5. Read how to load and run Code Example VBNET with this part.
'-----
```

### Get DimXpert Tolerance1 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Circularity geometric tolerance
- \* Cylindricity geometric tolerance
- \* Countersink angle dimension tolerance
- \* Countersink diameter dimension tolerance
- \* Flatness geometric tolerance
- \* Surface profile geometric tolerance
- \* Length dimension tolerance
- \* Width dimension tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.
// 2. Click Auto Dimension Scheme on the DimXpert toolbar and
// ensure that all feature filters are selected.
// Click the green check mark to accept the settings.
// 3. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// a. Select Cylindricity from the Symbol dropdown in the first row.
// b. In the second row, select Profile of Surface from the
// Symbol dropdown list and type a tolerance of 0.5 next to it.
// c. Click the outer-base cylinder and click again outside
// the part to place the annotation.
// d. Click OK to close the Geometric Tolerance Properties dialog.
// 4. In the DimXpertManager tab of the Management Panel, expand all of
// the nodes in the tree.
// 5. Observe the following DimXpert annotations: Flatness1, Circularity1,
// Diameter1, CounterSinkAngle1, CounterSinkDiameter1, Cylindricity1,
// Surface Profile1, Length1, Width1
// 6. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online tutorial,
// do not save any changes to the original file name.
// 7. Read how to load and run Code Example CSharp with this part.
//-----
```

### Get DimXpert Tolerance1 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Circularity geometric tolerance
- \* Cylindricity geometric tolerance
- \* Countersink angle dimension tolerance
- \* Countersink diameter dimension tolerance
- \* Flatness geometric tolerance
- \* Surface profile geometric tolerance
- \* Length dimension tolerance
- \* Width dimension tolerance

- 
1. Open:  
`public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt`
  2. Click **Auto Dimension Scheme** on the DimXpert toolbar and ensure that all feature filters are selected.  
Click the green check mark to accept the settings.
  3. Click **Geometric Tolerance** on the DimXpert toolbar.  
In the Geometric Tolerance Properties dialog:
    - a. Select **Cylindricity** from the **Symbol** dropdown list in the first row.
    - b. In the second row, select **Profile of Surface** from the **Symbol** dropdown list and type a tolerance of 0.5 next to it.
    - c. Click on the outer base cylinder and click again outside the part to place the annotation.
    - d. Click **OK** to close the Geometric Tolerance Properties dialog.
  4. In the DimXpertManager tab of the Management Panel, expand all of the nodes in the tree.
  5. Observe the following DimXpert annotations:  
**Flatness1, Circularity1, Diameter1, CounterSinkAngle1, CounterSinkDiameter1, Cylindricity1, Surface Profile1, Length1, Width1**
  6. Close the part, saving it to another name.  
NOTE: Because this part is used in a SOLIDWORKS online tutorial, do not save any changes to the original file name.
  7. Read how to load and run [Code Example VBNET](#) with this part.
-

## Get DimXpert Tolerance2 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Distance-between dimension tolerance
- \* Composite distance-between dimension tolerance
- \* Symmetry geometric tolerance
- \* Radius dimension tolerance
- \* Diameter dimension tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\api\block.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
// (select the first instance of toolbars on the View menu).
// 3. Create a distance-between dimension tolerance:
// a. Click Location Dimension on the DimXpert toolbar.
// b. Click the left front face of the block.
// c. Click Create Compound Plane on the DimXpert pop-up toolbar.
// d. Click the right front face of the block.
// e. Click the green check mark to accept the faces.
// f. Click the back face of the block to complete the
// location dimension.
// g. Click outside the part to position the location dimension.
// h. Click ESC to end dimensioning.
// i. Observe the following DimXpert annotation: DistanceBetween1
// 4. Create composite distance-between and radius dimension tolerances:
// a. Click Auto Dimension Scheme on the DimXpert toolbar.
// b. Select Part type: Prismatic.
// c. Select Tolerance type: Plus and Minus.
// d. Click in the Primary Datum field; select face in front of
// right hole.
// e. Click in the Secondary Datum field; select top face of block.
// f. Select all feature filters.
// g. Click the green check mark to accept the DimXpert settings.
// h. Observe the following DimXpert annotations: Diameter1,
// DistanceBetween5, Radius1.
// 5. Create a symmetry geometric tolerance:
// a. Click Datum on the DimXpert toolbar.
// b. Click the face in front of the left hole and click
// outside the part to place Datum A.
// c. Click the green check mark to finish the Datum definition.
// d. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Symmetry from the Symbol dropdown list.
// ii. Type A for the Primary Datum.
// iii. Click the face in front of the right hole.
// iv. Click outside the part to place the annotation.
// v. Click OK to close the Geometric Tolerance Properties dialog.
// e. Observe the following DimXpert annotations: Symmetry1.
// 6. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes to the original file name.
// 7. Read how to load and run Code Example CSharp with this part.
//-----
```

## Get DimXpert Tolerance2 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Distance-between dimension tolerance
- \* Composite distance-between dimension tolerance
- \* Symmetry geometric tolerance
- \* Radius dimension tolerance
- \* Diameter dimension tolerance

```

' -----
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Open the DimXpert toolbar from View > toolbars
' (select the first instance of toolbars on the View menu).
' 3. Create a distance-between dimension tolerance:
' a. Click Location Dimension on the DimXpert toolbar.
' b. Click the left-front face of the block.
' c. Click Create Compound Plane on the DimXpert
' pop-up toolbar.
' d. Click the right front face of the block.
' e. Click the green check mark to accept the faces.
' f. Click the back face of the block to complete the
' location dimension.
' g. Click outside the part to position the location dimension.
' h. Click ESC to end dimensioning.
' i. Observe the following DimXpert annotation: DistanceBetween1.
' 4. Create composite distance-between and radius dimension tolerances:
' a. Click Auto Dimension Scheme on the DimXpert toolbar.
' b. Select Part type: Prismatic.
' c. Select Tolerance type: Plus and Minus
' d. Click in the Primary Datum field; select face in front of right hole.
' e. Click in the Secondary Datum field; select top face of block.
' f. Select all feature filters.
' g. Click the green check mark to accept the DimXpert settings.
' h. Observe the following DimXpert annotations:
' Diameter1, DistanceBetween5, and Radius1.
' 5. Create a symmetry geometric tolerance:
' a. Click Datum on the DimXpert toolbar.
' b. Click the face in front of the left hole and click
' outside the part to place Datum A.
' c. Click the green check mark to finish the Datum
' definition.
' d. Click Geometric Tolerance on the DimXpert toolbar.
' In the Geometric Tolerance Properties dialog:
' i. Select Symmetry from the Symbol dropdown list.
' ii. Type A for the Primary Datum.
' iii. Click the face in front of the right hole.
' iv. Click outside the part to place the annotation.
' v. Click OK to close the Geometric Tolerance
' Properties dialog.
' e. Observe the following DimXpert annotation: Symmetry1.
' 6. Close the part, saving it to another name.
' NOTE: Because this part is used in a SOLIDWORKS online
' tutorial, do not save any changes to the original file name.
' 7. Read how to load and run Code Example VBNET with this part.
' -----

```

### Get DimXpert Tolerance3 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Position geometric tolerance
- \* Orientation (Perpendicularity) geometric tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\api\cover_position.sldprt.
// 2. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// a. Select Position from the Symbol dropdown
// list in the first row.
// b. Type A in Primary, B in Secondary, and C in Tertiary.
// c. In second row (lower tier), select Position from the
// Symbol dropdown list.
// d. Type 0.25 in Tolerance1.
// e. Type A in Primary.
// f. Select the Composite frame check box.
// g. Click the circular face of the boss.
// h. Click outside the part to place the annotation.
// i. Click OK to close the Geometric Tolerance
// Properties dialog.
// 3. Observe the following DimXpert annotations:
// Position1 and Perpendicularity1.
// 4. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes to the original file name.
// 5. Read how to load and run Code Example CSharp with this part.
//-----
```

### Get DimXpert Tolerance3 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Position geometric tolerance
- \* Orientation (Perpendicularity) geometric tolerance

- ```
' -----  
' 1. Open public_documents\samples\tutorial\api\cover_position.sldprt.  
' 2. Click Geometric Tolerance on the DimXpert toolbar.  
'   In the Geometric Tolerance Properties dialog:  
'   a. Select Position from the Symbol dropdown list in the first row.  
'   b. Type A in Primary, B in Secondary, and C in Tertiary.  
'   c. In second row (lower tier), select Position  
'      from the Symbol dropdown list.  
'   d. Type 0.25 in Tolerance1.  
'   e. Type A in Primary.  
'   f. Select the Composite frame check box.  
'   g. Click the circular face of the boss.  
'   h. Click outside the part to place the annotation.  
'   i. Click OK to close the Geometric Tolerance  
'      Properties dialog.  
' 3. Observe the following DimXpert annotations:  
'   Position1 and Perpendicularity1.  
' 4. Close the part, saving it to another name.  
'   NOTE: Because this part is used in a SOLIDWORKS online  
'        tutorial, do not save any changes to the original  
'        file name.  
' 5. Read how to load and run Code Example VBNET with this part.  
' -----
```

Get DimXpert Tolerance4 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Angle-between dimension tolerance
- * Chamfer dimension tolerance
- * Line Profile geometric tolerance
- * Straightness geometric tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
//    (select the first instance of Toolbars on the View menu).
// 3. Create the chamfer dimension tolerance:
//    a. Click Auto-Dimension Scheme on the DimXpert toolbar.
//    b. Ensure that the Chamfer feature filter is selected.
//    c. Click the green arrow to accept the settings.
//    d. Observe ChamferDepth1 and ChamferDepth2 in the DimXpert tree.
// 4. Create angle-between dimension tolerance:
//    a. Click Location Dimension on the DimXpert toolbar.
//    b. Click a face of the part.
//    c. Click a face perpendicular to the face selected
//       in step 2 (not the chamfer).
//    d. Click outside the part to place the annotation.
//    e. Find the following in the DimXpert tree: AngleBetween1.
// 5. Create line profile and straightness geometric tolerances:
//    a. Click Geometric Tolerance on the DimXpert toolbar.
//       In the Geometric Tolerance Properties dialog:
//       i. Select Straightness from the Symbol dropdown list.
//       ii. In second row, select Line Profile from the Symbol
//          dropdown list.
//       iii. Type 0.25 in Tolerance1.
//       iv. Click the top face of the part.
//       v. Click outside the part to place the annotation.
//       vi. Click OK to close the Geometric Tolerance
//          Properties dialog.
//    b. Observe the following DimXpert annotations:
//       LineProfile1 and Straightness1.
// 6. Close the part, saving it to another name.
//    NOTE: Because these parts are used in a
//          SOLIDWORKS online tutorial, do not
//          save any changes to the original file name.
// 7. Read how to load and run Code Example CSharp with this part.
//-----
```

Get DimXpert Tolerance4 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Angle-between dimension tolerance
- * Chamfer dimension tolerance
- * Line Profile geometric tolerance
- * Straightness geometric tolerance

1. Open `public_documents\samples\tutorial\api\plate_tolstatus.sldprt`.
2. Open the DimXpert toolbar from **View > Toolbars**
(select the first instance of Toolbars on the View menu).
3. Create the chamfer dimension tolerance:
 - a. Click **Auto-Dimension Scheme** on the DimXpert toolbar.
 - b. Ensure that the Chamfer feature filter is selected.
 - c. Click the green arrow to accept the settings.
 - d. Observe **ChamferDepth1** and **ChamferDepth2** in the DimXpert tree.
4. Create angle-between dimension tolerance:
 - a. Click **Location Dimension** on the DimXpert toolbar.
 - b. Click a face of the part.
 - c. Click a face perpendicular to the face
selected in step 2 (not the chamfer).
 - d. Click outside the part to place the annotation.
 - e. Find the following in the DimXpert tree: **AngleBetween1**.
5. Create line profile and straightness geometric tolerances:
 - a. Click **Geometric Tolerance** on the DimXpert toolbar.
In the Geometric Tolerance Properties dialog:
 - i. Select **Straightness** from the **Symbol** dropdown
list.
 - ii. In second row, select **Profile of Line** from
the **Symbol** dropdown list.
 - iii. Type **0.25** in **Tolerance1**.
 - iv. Click the top face of the part.
 - v. Click outside the part to place the
annotation.
 - vi. Click **OK** to close the Geometric Tolerance
Properties dialog.
 - b. Observe the following DimXpert annotations:
LineProfile1 and **Straightness1**.
6. Close the part, saving it to another name.
NOTE: Because these parts are used in a SOLIDWORKS
online tutorial, do not save any changes to
the original file name.
7. Read how to load and run [Code Example VBNET](#) with this part.

Get DimXpert Tolerance5 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert concentricity geometric tolerance annotation.

```
//-----  
// 1. Open public_documents\samples\tutorial\api\cover_with_dimensions.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
//    (select the first instance of Toolbars on the View menu).  
// 3. Create concentricity tolerance:  
//    a. Click Datum on the DimXpert toolbar.  
//    b. Click Boss1 (front extrusion).  
//    c. Click outside the part to place the annotation.  
//    d. Click the green check mark to accept Datum A.  
//    e. Click Geometric Tolerance on the DimXpert toolbar.  
//        In the Geometric Tolerance Properties dialog:  
//        i.    Select Concentricity from the Symbol dropdown list.  
//        ii.   Click inside the Tolerance 1 field.  
//        iii.  Click Diameter on the toolbar  
//            at the top of the dialog.  
//        iv.   Type A in the Primary field.  
//        v.    Click the inside face of Simple Hole2.  
//        vi.   Click outside the part to place the annotation.  
//        vii.  Click OK to close the Geometric Tolerance Properties dialog.  
//    f. In the DimXpertManager tab of the Management Panel,  
//        expand Simple Hole2.  
//    g. Observe the following Dimxpert annotation: Concentricity1.  
// 4. Close the part, saving it to another name.  
//    NOTE: Because this part is used in a SOLIDWORKS online tutorial,  
//        do not save any changes to the original file name.  
// 5. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get DimXpert Tolerance5 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert concentricity geometric tolerance annotation.

```
'-----  
' 1. Open public_documents\samples\tutorial\api\cover_with_dimensions.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars  
'    (select the first instance of Toolbars on the View menu).  
' 3. Create concentricity tolerance:  
'    a. Click Datum on the DimXpert toolbar.  
'    b. Click Boss1 (front extrusion).  
'    c. Click outside the part to place the annotation.  
'    d. Click the green check mark to accept Datum A.  
'    e. Click Geometric Tolerance on the DimXpert toolbar.  
'        In the Geometric Tolerance Properties dialog:  
'            i. Select Concentricity from the Symbol dropdown list.  
'            ii. Click inside the Tolerance 1 field.  
'            iii. Click Diameter on the toolbar  
'                  at the top of the dialog.  
'            iv. Type A in the Primary field.  
'            v. Click the inside face of Simple Hole2.  
'            vi. Click outside the part to place the annotation.  
'            vii. Click OK to close the Geometric Tolerance  
'                  Properties dialog.  
'    f. In the DimXpertManager tab of the Management Panel,  
'        expand Simple Hole2.  
'    g. Observe the following Dimxpert annotation: Concentricity1.  
' 4. Close the part, saving it to another name.  
'    NOTE: Because this part is used in a SOLIDWORKS online  
'          tutorial, do not save any changes to the original  
'          file name.  
' 5. Read how to load and run Code Example VBNET with this part.  
'-----
```

Get DimXpert Tolerance6 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Surface profile geometric tolerance
- * Tangency geometric tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\cosmosxpress\aw_anchor_plate.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
//    (select the first instance of Toolbars on the View menu).
// 3. Create a surface profile tolerance:
//    a. Click Datum on the DimXpert toolbar.
//    b. Click a top face of the part.
//    c. Click outside the part to place Datum A.
//    d. Click a back face of the part.
//    e. Click outside the part to place Datum B.
//    f. Click a side face of the part.
//    g. Click outside the part to place Datum C.
//    h. Click Geometric Tolerance on the DimXpert toolbar.
//        In the Geometric Tolerance Properties dialog:
//        i.   Select Profile of Surface from the Symbol dropdown list.
//        ii.  Type 50 in Tolerance 1.
//        iii. Type A in the Primary box and C in the
//            Secondary box.
//        iv.  In the second row, select Profile of Surface from
//            the Symbol dropdown list.
//        v.   In the second row, type 50 in Tolerance 1.
//        vi.  In the second row, type A in the Primary box.
//        vii. Select the Composite frame check box.
//        viii. Click the front curve of the hook.
//        ix.  Click outside the part to place the annotation.
//        x.   Click OK to close the Geometric Tolerance
//            Properties dialog.
//    i. In the DimXpertManager tab of the Management Panel,
//        expand Cylinder1.
//    j. Observe the following DimXpert annotation:
//        Surface Profile1 and Surface Profile2
// 4. Create tangency tolerances:
//    a. Click Auto Dimension Scheme on the DimXpert toolbar.
//    b. Select all feature filters.
//    c. Click the green check mark to accept the settings.
//    d. Observe several tangency tolerance annotations.
// 5. Close the part, saving it to another name.
//    NOTE: Because this part is used in a SOLIDWORKS online
//          tutorial, do not save any changes to the original
//          file name.
// 6. Read how to load and run Code Example CSharp with this part.
//-----
```

Get DimXpert Tolerance6 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Surface profile geometric tolerance
- * Tangency geometric tolerance

```

' -----
' 1. Open public_documents\samples\tutorial\cosmosxpress\aw_anchor_plate.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
'    (select the first instance of Toolbars on the View menu).
' 3. Create a surface profile tolerance:
'    a. Click Datum on the DimXpert toolbar.
'    b. Click a top face of the part.
'    c. Click outside the part to place Datum A.
'    d. Click a back face of the part.
'    e. Click outside the part to place Datum B.
'    f. Click a side face of the part.
'    g. Click outside the part to place Datum C.
'    h. Click Geometric Tolerance on the DimXpert toolbar.
'       In the Geometric Tolerance Properties dialog:
'       i.   Select Profile of Surface from the Symbol dropdown list.
'       ii.  Type 50 in Tolerance 1.
'       iii. Type A in the Primary box and C in the
'           Secondary box.
'       iv.  In the second row, select Profile of Surface from
'           the Symbol dropdown list.
'       v.   In the second row, type 50 in Tolerance 1.
'       vi.  In the second row, type A in the Primary box.
'       vii. Select the Composite frame check box.
'       viii. Click the front curve of the hook.
'       ix.  Click outside the part to place the annotation.
'       x.   Click OK to close the Geometric Tolerance
'           Properties dialog.
'    i. In the DimXpertManager tab of the Management Panel,
'       expand Cylinder1.
'    j. Observe the following DimXpert annotations:
'       Surface Profile1 and Surface Profile2
' 4. Create tangency tolerances:
'    a. Click Auto Dimension Scheme on the DimXpert toolbar.
'    b. Select all feature filters.
'    c. Click the green check mark to accept the settings.
'    d. Observe several tangency tolerance annotations.
' 5. Close the part, saving it to another name.
'    NOTE: Because this part is used in a SOLIDWORKS online
'          tutorial, do not save any changes to the original
'          file name.
' 6. Read how to load and run Code Example VBNET with this part.
' -----

```

Get DimXpert Tolerance7 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert line profile geometric tolerance annotation.

```
//-----  
// 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
//    (select the first instance of Toolbars on the View menu).  
// 3. Click Datum on the DimXpert toolbar and create  
//    Datum A somewhere on the part.  
// 4. Create the line profile geometric tolerance:  
//    a. Click Geometric Tolerance on the DimXpert toolbar.  
//    In the Geometric Tolerance Properties dialog:  
//        i.    Select Line Profile from the Symbol dropdown in  
//             the first row.  
//        ii.   Type A in Primary of the first row.  
//        iii.  In the second row, select Line Profile from the Symbol  
//             dropdown list.  
//        iv.   In the second row, type a tolerance in Tolerance 1.  
//        v.    In the second row, type A in Primary.  
//        vi.   Select Composite frame.  
//        vii.  Click the inner face of the hole.  
//        viii. Click outside the part to place the annotation.  
//        ix.   Click OK to close the Geometric Tolerance  
//             Properties dialog.  
//    b. Observe LineProfile1 and LineProfile2 under SimpleHole1  
//        in the DimXpert tree.  
// 5. Close the part, saving it to another name.  
//    NOTE: Because these parts are used in a SOLIDWORKS online  
//          tutorial, do not save any changes to the original  
//          file name.  
// 6. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get DimXpert Tolerance7 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for a DimXpert line profile geometric tolerance annotation.

- ```
'-----
' 1. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
' (select the first instance of Toolbars on the View menu).
' 3. Click Datum on the DimXpert toolbar and create
' Datum A somewhere on the part.
' 4. Create the line profile geometric tolerance:
' a. Click Geometric Tolerance on the DimXpert toolbar.
' In the Geometric Tolerance Properties dialog:
' i. Select Line Profile from the Symbol dropdown in
' the first row.
' ii. Type A in Primary of the first row.
' iii. In the second row, select Line Profile from the Symbol
' dropdown list.
' iv. In the second row, type a tolerance in Tolerance 1.
' v. In the second row, type A in Primary.
' vi. Select Composite frame.
' vii. Click the inner face of the hole.
' viii. Click outside the part to place the annotation.
' ix. Click OK to close the Geometric Tolerance
' Properties dialog.
' b. Observe LineProfile1 and LineProfile2 under SimpleHole1
' in the DimXpert tree.
' 5. Close the part, saving it to another name.
' NOTE: Because these parts are used in a SOLIDWORKS online
' tutorial, do not save any changes to the original
' file name.
' 6. Read how to load and run Code Example VBNET with this part.
'-----
```

## Get DimXpert Tolerance8 Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Circular runout geometric tolerance
- \* Total runout geometric tolerance
- \* Angularity geometric tolerance
- \* Parallelism geometric tolerance

```
//-----
// 1. Open public_documents\samples\tutorial\api\cover_datum.sldprt.
// 2. Click Datum on the DimXpert toolbar and place
// Datum A inside the hole.
// 3. Create the circular runout geometric tolerance:
// a. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Circular Runout from the Symbol dropdown
// list in the first row.
// ii. In the Primary field, type A.
// iii. Click the outer cylinder of the part and click
// again outside the part to place
// the annotation.
// iv. Click OK to close the Geometric Tolerance
// Properties dialog.
// b. Observe CircularRunout1 under Cylinder1 in the
// DimXpert tree.
// 4. Create the total runout geometric tolerance:
// a. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Total Runout from the Symbol dropdown
// list in the first row.
// ii. In the Primary field, type A.
// iii. Click the boss of the part and click again
// outside the part to place
// the annotation.
// iv. Click OK to close the Geometric Tolerance
// Properties dialog.
// b. Observe TotalRunout1 under Boss1 in the DimXpert
// tree.
// 5. Create the angularity geometric tolerance:
// a. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Angularity from the Symbol dropdown
// in the first row.
// ii. In the Primary field, type A.
// iii. Click a face of the notch and click again
// outside the part to place the annotation.
// iv. Click OK to close the Geometric Tolerance
// Properties dialog.
// b. Observe Angularity1 under Notch1 in the DimXpert
// tree.
// 6. Create the parallelism geometric tolerance:
// a. Click Geometric Tolerance on the DimXpert toolbar.
// In the Geometric Tolerance Properties dialog:
// i. Select Parallel from the Symbol dropdown
// in the first row.
// ii. In the Primary field, type A.
// iii. Click the boss of the part and click again
// outside the part to place the annotation.
// iv. Click OK to close the Geometric Tolerance
// Properties dialog.
```

```
// b. Observe Parallelism1 under Boss1 in the DimXpert tree.
// 7. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes to the original
// file name.
// 8. Read how to load and run Code Example CSharp with this part.
//-----
```

## Get DimXpert Tolerance8 Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- \* Circular runout geometric tolerance
- \* Total runout geometric tolerance
- \* Angularity geometric tolerance
- \* Parallelism geometric tolerance

- ```

'-----
' 1. Open public_documents\samples\tutorial\api\cover_datum.sldprt.
' 2. Click Datum on the DimXpert toolbar and place
'    Datum A inside the hole.
' 3. Create the circular runout geometric tolerance:
'    a. Click Geometric Tolerance on the DimXpert toolbar.
'       In the Geometric Tolerance Properties dialog:
'       i. Select Circular Runout from the Symbol dropdown
'          list in the first row.
'       ii. In the Primary field, type A.
'       iii. Click the outer cylinder of the part and click
'            again outside the part to place
'            the annotation.
'       iv. Click OK to close the Geometric Tolerance
'            Properties dialog.
'    b. Observe CircularRunout1 under Cylinder1 in the
'       DimXpert tree.
' 4. Create the total runout geometric tolerance:
'    a. Click Geometric Tolerance on the DimXpert toolbar.
'       In the Geometric Tolerance Properties dialog:
'       i. Select Total Runout from the Symbol dropdown
'          list in the first row.
'       ii. In the Primary field, type A.
'       iii. Click the boss of the part and click again
'            outside the part to place
'            the annotation.
'       iv. Click OK to close the Geometric Tolerance
'            Properties dialog.
'    b. Observe TotalRunout1 under Boss1 in the DimXpert
'       tree.
' 5. Create the angularity geometric tolerance:
'    a. Click Geometric Tolerance on the DimXpert toolbar.
'       In the Geometric Tolerance Properties dialog:
'       i. Select Angularity from the Symbol dropdown
'          in the first row.
'       ii. In the Primary field, type A.
'       iii. Click a face of the notch and click again
'            outside the part to place the annotation.
'       iv. Click OK to close the Geometric Tolerance
'            Properties dialog.
'    b. Observe Angularity1 under Notch1 in the DimXpert
'       tree.
' 6. Create the parallelism geometric tolerance:
'    a. Click Geometric Tolerance on the DimXpert toolbar.
'       In the Geometric Tolerance Properties dialog:
'       i. Select Parallel from the Symbol dropdown
'          in the first row.
'       ii. In the Primary field, type A.
'       iii. Click the boss of the part and click again
'            outside the part to place the annotation.
'       iv. Click OK to close the Geometric Tolerance
'            Properties dialog.

```

' b. Observe **Parallelism1** under **Boss1** in the DimXpert tree.

' 7. Close the part, saving it to another name.

' NOTE: Because this part is used in a SOLIDWORKS online

' tutorial, do not save any changes to the original

' file name.

' 8. Read how to load and run [Code Example_VBNET](#) with this part.

' -----

Get DimXpert Tolerance Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Counterbore dimension tolerance
- * Depth dimension tolerance

```
//-----  
// 1. Open public_documents\samples\tutorial\api\face_plate_ads_geo.sldprt.  
// 2. Open the DimXpert toolbar from View > Toolbars  
//    (select the first instance of Toolbars on the View menu).  
// 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
// 4. Ensure that all feature filters are selected.  
// 5. Click the green check mark to accept the settings.  
// 6. In the DimXpertManager tab of the Management Panel,  
//    expand Hole Pattern2 in the DimXpertManager tab of the Management  
//    Panel.  
// 7. Observe the following DimXpert annotations:  
//    Counterbore1 and Depth1.  
// 8. Close the part, saving it to another name.  
//    NOTE: Because this part is used in a SOLIDWORKS  
//          online tutorial, do not save any changes to  
//          the original file name.  
// 9. Read how to load and run Code Example CSharp with this part.  
//-----
```

Get DimXpert Tolerance Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert annotations:

- * Counterbore dimension tolerance
- * Depth dimension tolerance

- ```
' -----
' 1. Open public_documents\samples\tutorial\api\face_plate_ads_geo.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
' (select the first instance of Toolbars on the View menu).
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.
' 4. Ensure that all feature filters are selected.
' 5. Click the green check mark to accept the settings.
' 6. In the DimXpertManager tab of the Management Panel,
' expand Hole Pattern2 in the DimXpertManager tab of the Management
' Panel.
' 7. Observe the following DimXpert annotations:
' Counterbore1 and Depth1.
' 8. Close the part, saving it to another name.
' NOTE: Because this part is used in a SOLIDWORKS
' online tutorial, do not save any changes to
' the original file name.
' 9. Read how to load and run Code Example VBNET with this part.
' -----
```

### Get DimXpert Compound Width and Best Fit Plane Features Example (C#)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- \* Width
- \* Best fit plane

```
//-----
// 1. Open public_documents\samples\tutorial\api\block.sldprt.
// 2. Open the DimXpert toolbar from View > Toolbars
// (select the first instance of Toolbars on the View menu).
// 3. Create the DimXpert best fit plane feature:
// a. Click Location Dimension on the DimXpert toolbar.
// b. Select the left-front face of the block.
// c. Click Compound Plane on the DimXpert pop-up
// toolbar.
// d. Select the right front face of the block.
// e. Click the green check mark on the DimXpert pop-up toolbar.
// f. Select the back face of the block.
// g. Click outside the part to place the location dimension
// annotation.
// h. Observe the best fit plane DimXpert feature on the
// DimXpertManager tab.
// 4. Create the DimXpert width feature:
// a. Click Size Dimension on the DimXpert toolbar.
// b. Select a front face of the block.
// c. Click Width on the DimXpert pop-up toolbar.
// d. Select the back face of the block.
// e. Click the green check mark on the DimXpert pop-up toolbar.
// f. Click outside the part to place the size dimension annotation.
// 5. Observe the following DimXpert feature on the DimXpertManager
// tab: Width1.
// 6. Close the part, saving it to another name.
// NOTE: Because this part is used in a SOLIDWORKS online
// tutorial, do not save any changes to the original
// file name.
// 7. Read how to load and run Code Example CSharp with this part.
//-----
```

Get DimXpert Compound Width and Best Fit Plane Features Example (VB.NET)

This example builds a part to demonstrate usage of the SOLIDWORKS Document Manager API for the following DimXpert features:

- \* Width
- \* Best fit plane

```
'-----
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Open the DimXpert toolbar from View > Toolbars
' (select the first instance of Toolbars on the View menu).
' 3. Create the DimXpert best fit plane feature:
' a. Click Location Dimension on the DimXpert toolbar.
' b. Select the left-front face of the block.
' c. Click Compound Plane on the DimXpert pop-up
' toolbar.
' d. Select the right front face of the block.
' e. Click the green check mark on the DimXpert pop-up toolbar.
' f. Select the back face of the block.
' g. Click outside the part to place the location dimension
' annotation.
' h. Observe the best fit plane DimXpert feature on the
' DimXpertManager tab.
' 4. Create the DimXpert width feature:
' a. Click Size Dimension on the DimXpert toolbar.
' b. Select a front face of the block.
' c. Click Width on the DimXpert pop-up toolbar.
' d. Select the back face of the block.
' e. Click the green check mark on the DimXpert pop-up toolbar.
' f. Click outside the part to place the size dimension annotation.
' 5. Observe the following DimXpert feature on the DimXpertManager
' tab: Width1.
' 6. Close the part, saving it to another name.
' NOTE: Because this part is used in a SOLIDWORKS online
' tutorial, do not save any changes to the original
' file name.
' 7. Read how to load and run Code Example VBNET with this part.
'-----
```

## Get Display Title Name (C#)

This example shows how to get the display title name of a document.

```
// =====
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
// topic and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
// in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Select Add Reference.
// c. Click Browse.
// d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
// e. Click Add.
// f. Click Close.
// 4. Ensure that sLicenseKey and sDocFileName point to valid strings.
// 5. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
// =====
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Security;
using System.Text;
using System.Threading.Tasks;
using Microsoft.VisualBasic;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swdocumentmgr;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

partial class SolidWorksMacro
{
 public void main()
 {
 SwDMClassFactory swClassFact;
 SwDMApplication swDocMgr;
 SwDMDocument25 swDoc;
 string sLicenseKey;
 sLicenseKey = "your_license_key";
 string sDocFileName;
 sDocFileName = "doc_file_name";
 int nDocType;
 nDocType = SwDmDocumentType.swDmDocumentPart;
 swClassFact = new SwDMClassFactory();
 swDocMgr = swClassFact.GetApplication(sLicenseKey);
 SwDmDocumentOpenError retval;
 swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, false, retval);
 Debug.Print("File = " + swDoc.FullName);
 Debug.Print("DisplayTitleName = " + swDoc.GetDisplayTitleName());

 swDoc.CloseDoc();
 }
}
```

```
/// <summary>
/// ''' The SldWorks swApp variable is pre-assigned for you.
/// ''' </summary>
public SldWorks swApp;
}
```

## Get Document Settings Example (C#)

This example shows how to get various settings for a document.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
// topic and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
// in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Select Add Reference.
// c. Click Browse.
// d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
// e. Click Add.
// f. Click Close.
// 4. Substitute path_to_SOLIDWORKS_2015_model with the path to a model that has been
// saved in SOLIDWORKS 2015 or later.
// 5. Substitute your_license_key with your SOLIDWORKS Document Manager
// license key.
// 6. Open the Immediate window.
//
// Postconditions: Examine the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save any changes.
//-----

using System;
using System.Collections.Generic;
using System.Text;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;

namespace GetExtRefs2_CSharp
{
 class Program
 {
 static SwDMClassFactory dmClassFact;
 static SwDMAApplication4 dmDocMgr;
 static SwDMDocument19 dmDoc;
 static SwDMDocumentType dmDocType;
 static SwDMDocumentOpenError status;

 const string docPath = "path_to_SOLIDWORKS_2015_model";
 const string licenseKey = "your_license_key";

 static void Main()
 {
 dmClassFact = new SwDMClassFactory();
 dmDocMgr = (SwDMAApplication4)dmClassFact.GetApplication(licenseKey);
 dmDoc = (SwDMDocument19)dmDocMgr.GetDocument(docPath, dmDocType, true, out status);

 if ((dmDoc != null))
 {
 Debug.Print("Angular units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsAngular));
 Debug.Print("Angular decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsAngularDecimals));
 Debug.Print("Decimal rounding: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsDecimalRounding));
 Debug.Print("Dual linear units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsDualLinear));
 Debug.Print("Dual linear decimal display: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsDualLinearDecimals));
 Debug.Print("Dual linear decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsDualLinearDecimals));
 Debug.Print("Dual linear fraction denominator: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsDualLinearFractionDenominator));
 Debug.Print("Energy decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsEnergyDecimals));
 Debug.Print("Energy units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsEnergyUnits));
 Debug.Print("Force units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsForceUnits));
 Debug.Print("Force decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsForceDecimals));
 Debug.Print("Linear units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsLinear));
 Debug.Print("Linear decimal display: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsLinearDecimals));
 Debug.Print("Linear decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsLinearDecimals));
 Debug.Print("Linear fraction denominator: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsLinearFractionDenominator));
 Debug.Print("Mass prop decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropDecimals));
 Debug.Print("Mass prop length: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropLength));
 Debug.Print("Mass prop mass: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropMass));
 Debug.Print("Mass prop volume: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropVolume));
 Debug.Print("Power decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsPowerDecimals));
 Debug.Print("Power units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsPowerUnits));
 Debug.Print("Time decimal places: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsTimeDecimals));
 Debug.Print("Time units: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsTimeUnits));
 Debug.Print("Unit system: " + dmDoc.GetSwDmSettingInteger((int)swDmDocumentUnitsIntegerValue_e.swDmUnitsUnitSystem));

 Debug.Print("Dual linear feet and inches format? " + dmDoc.GetSwDmSettingToggle((int)swDmDocumentUnitsToggle_e.swDmUnitsDualLinearFeetAndInchesFormat));
 Debug.Print("Dual linear round to nearest fraction? " + dmDoc.GetSwDmSettingToggle((int)swDmDocumentUnitsToggle_e.swDmUnitsDualLinearRoundToNearestFraction));
 Debug.Print("Linear feet and inches format? " + dmDoc.GetSwDmSettingToggle((int)swDmDocumentUnitsToggle_e.swDmUnitsLinearFeetAndInchesFormat));
 Debug.Print("Linear round to nearest fraction? " + dmDoc.GetSwDmSettingToggle((int)swDmDocumentUnitsToggle_e.swDmUnitsLinearRoundToNearestFraction));
 }
 }
 }
}
```

```
 else
 {
 Debug.Print("Unable to open document. Check docPath variable.");
 }
 }
}
```

## Get Document Settings Example (VB.NET)

This example shows how to get various settings for a document.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started
' topic and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
' in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
' a. Right-click the solution in Solution Explorer.
' b. Select Add Reference.
' c. Click Browse.
' d. Click install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll.
' e. Click Add.
' f. Click Close.
' 4. Substitute path_to_SOLIDWORKS_2015_model with the path to a model that has been
' saved in SOLIDWORKS 2015 or later.
' 5. Substitute your_license_key with your SOLIDWORKS Document Manager
' license key.
' 6. Open the Immediate window.
'
' Postconditions: Examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save any changes.
'-----

```

```

Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

```

```
Module Module1
```

```

 Dim dmClassFact As SwDMClassFactory
 Dim dmDocMgr As SwDMApplication4
 Dim dmDoc As SwDMDocument19
 Dim dmDocType As SwDMDocumentType
 Dim status As SwDMDocumentOpenError

```

```

 Const docPath As String = "path_to_SOLIDWORKS_2015_model"
 Const licenseKey As String = "your_license_key"

```

```
Sub main()
```

```

 dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 dmDocMgr = dmClassFact.GetApplication(licenseKey)
 dmDoc = dmDocMgr.GetDocument(docPath, dmDocType, True, status)

```

```

 If Not (dmDoc Is Nothing) Then
 Debug.Print("Angular units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsAngular))
 Debug.Print("Angular decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsAngularDec
 Debug.Print("Decimal rounding: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsDecimalRounding)
 Debug.Print("Dual linear units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsDualLinear))
 Debug.Print("Dual linear decimal display: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinea
 Debug.Print("Dual linear decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinear
 Debug.Print("Dual linear fraction denominator: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnits
 Debug.Print("Energy decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsEnergyDecin
 Debug.Print("Energy units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsEnergyUnits))
 Debug.Print("Force units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsForce))
 Debug.Print("Force decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsForceDecimal
 Debug.Print("Linear units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinear))
 Debug.Print("Linear decimal display: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinearDeci
 Debug.Print("Linear decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinearDecin
 Debug.Print("Linear fraction denominator: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsLinea
 Debug.Print("Mass prop decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsMassProp
 Debug.Print("Mass prop length: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropLength))
 Debug.Print("Mass prop mass: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropMass))
 Debug.Print("Mass prop volume: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsMassPropVolume))
 Debug.Print("Power decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsPowerDecimal
 Debug.Print("Power units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsPowerUnits))
 Debug.Print("Time decimal places: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsTimeDecimalPl
 Debug.Print("Time units: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsTimeUnits))
 Debug.Print("Unit system: " & dmDoc.GetSwDmSettingInteger(swDmDocumentUnitsIntegerValue_e.swDmUnitsUnitSystem))

 Debug.Print("Dual linear feet and inches format? " & dmDoc.GetSwDmSettingToggle(swDmDocumentUnitsToggle_e.swDmUnitsDualL
 Debug.Print("Dual linear round to nearest fraction? " & dmDoc.GetSwDmSettingToggle(swDmDocumentUnitsToggle_e.swDmUnitsDu
 Debug.Print("Linear feet and inches format? " & dmDoc.GetSwDmSettingToggle(swDmDocumentUnitsToggle_e.swDmUnitsLinearFeet
 Debug.Print("Linear round to nearest fraction? " & dmDoc.GetSwDmSettingToggle(swDmDocumentUnitsToggle_e.swDmUnitsLinearF

```

```

 Else
 Debug.Print("Unable to open document. Check docPath variable.")
 End If

```

```
End Sub
```

```
End Module
```

## Get Drawing Sheets' Properties Example (C#)

This example shows how to get the properties of a drawing's sheets.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic
// and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
// in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference
// to the project:
// a. Right-click the solution in Solution Explorer.
// b. Click Add Reference.
// c. Click Browse.
// d. Click:
// install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 4. Ensure that the specified drawing document exists.
// 5. Open the Immediate window.
// 6. Substitute your_license_key with your SOLIDWORKS Document
// Manager license key.
// 7. Run the macro.
//
// Postconditions: Examine the Immediate window.
//-----

using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using SolidWorks.Interop.swdocumentmgr;

class Program
{
 static void Main(string[] args) {
 SwDMClassFactory swClassFact;
 SwDMAApplication docMgrApp;
 SwDMDocument13 docDrw;
 SwDmDocumentOpenError res;

 swClassFact = new SwDMClassFactory();
 docMgrApp = (SwDMAApplication)swClassFact.GetApplication("your_license_key");
 string sDocFileName;
 sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";
 docDrw = (SwDMDocument13)docMgrApp.GetDocument(sDocFileName, SwDmDocumentType.swDmDocumentDrawing, true, out res);
 string[] sheetNameVar;
 int sheetCount;

 sheetCount = docDrw.GetSheetCount();
 sheetNameVar = (string[])docDrw.GetSheetNames();

 string sheetName;
 int sheetNameCount;
 for (sheetNameCount = 0; sheetNameCount <= sheetNameVar.Length - 1; sheetNameCount++)
 {
 string formatName;
 swSheetFormatPathResult status1;
 sheetName = sheetNameVar[sheetNameCount];
 status1 = (swSheetFormatPathResult)docDrw.GetSheetFormatPath(sheetName, out formatName);

 Debug.Print(sheetName + " ---- " + formatName + "\n");

 swSheetPropertiesResult status2;
 object formatPropObj;

 status2 = (swSheetPropertiesResult)docDrw.GetSheetProperties(sheetName, out formatPropObj);

 double[] formatProp = (double[])formatPropObj;

 double para;
 para = System.Math.Round(formatProp[0]);

 string paperSize;

 GetPaperSize(para, out paperSize);

 Debug.Print("Paper Size = " + paperSize);
 Debug.Print("Size = " + formatProp[1] + " x " + formatProp[2]);
 Debug.Print("Scale = " + formatProp[3] + " : " + formatProp[4]);

 bool boolData;

 boolData = false;
 }
 }
}
```

```

 if ((formatProp[5] == 1))
 {
 boolData = true;
 }

 Debug.Print("First angle = " + boolData);
 string templateSize;
 para = System.Math.Round(formatProp[6]);
 GetTemplateSize(para, out templateSize);
 Debug.Print("Template size = " + templateSize);
 boolData = false;

 if ((formatProp[7] == 1))
 {
 boolData = true;
 }

 Debug.Print("Template visible = " + boolData);
 Debug.Print("=====");
 }
 docDrw.CloseDoc();
}

```

```

static void GetPaperSize(double inpNum, out string outPaperType)
{
 if ((inpNum == 0))
 {
 outPaperType = "swDwgPaperAsize";
 }
 else if ((inpNum == 1))
 {
 outPaperType = "swDwgPaperAsizeVertical";
 }
 else if ((inpNum == 2))
 {
 outPaperType = "swDwgPaperBsize";
 }
 else if ((inpNum == 3))
 {
 outPaperType = "swDwgPaperCsize";
 }
 else if ((inpNum == 4))
 {
 outPaperType = "swDwgPaperDsize";
 }
 else if ((inpNum == 5))
 {
 outPaperType = "swDwgPaperEsize";
 }
 else if ((inpNum == 6))
 {
 outPaperType = "swDwgPaperA4size";
 }
 else if ((inpNum == 7))
 {
 outPaperType = "swDwgPaperA4sizeVertical";
 }
 else if ((inpNum == 8))
 {
 outPaperType = "swDwgPaperA3size";
 }
 else if ((inpNum == 9))
 {
 outPaperType = "swDwgPaperA2size";
 }
 else if ((inpNum == 10))
 {
 outPaperType = "swDwgPaperA1size";
 }
 else if ((inpNum == 11))
 {
 outPaperType = "swDwgPaperA0size";
 }
 else
 {
 outPaperType = "swDwgPapersUserDefined";
 }
}

```

```

static void GetTemplateSize(double inpNum, out string outTemplateType)
{
 if ((inpNum == 0))
 {
 outTemplateType = "swDwgTemplateAsize";
 }
 else if ((inpNum == 1))
 {
 outTemplateType = "swDwgTemplateAsizeVertical";
 }
 else if ((inpNum == 2))
 {
 outTemplateType = "swDwgTemplateBsize";
 }
}

```

```
 else if ((inpNum == 3))
 {
 outTemplateType = "swDwgTemplateCsize";
 }
 else if ((inpNum == 4))
 {
 outTemplateType = "swDwgTemplateDsize";
 }
 else if ((inpNum == 5))
 {
 outTemplateType = "swDwgTemplateEsize";
 }
 else if ((inpNum == 6))
 {
 outTemplateType = "swDwgTemplateA4size";
 }
 else if ((inpNum == 7))
 {
 outTemplateType = "swDwgTemplateA4sizeVertical";
 }
 else if ((inpNum == 8))
 {
 outTemplateType = "swDwgTemplateA3size";
 }
 else if ((inpNum == 9))
 {
 outTemplateType = "swDwgTemplateA2size";
 }
 else if ((inpNum == 10))
 {
 outTemplateType = "swDwgTemplateA1size";
 }
 else if ((inpNum == 11))
 {
 outTemplateType = "swDwgTemplateA0size";
 }
 else if ((inpNum == 12))
 {
 outTemplateType = "swDwgTemplateCustom";
 }
 else
 {
 outTemplateType = "swDwgTemplateNone";
 }
}
}
```

## Get Drawing Sheets' Properties Example (VB.NET)

This example shows how to get the properties of a drawing's sheets.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API Getting Started topic
' and ensure that the required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
' in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference
' to the project:
' a. Right-click the solution in Solution Explorer.
' b. Click Add Reference.
' c. Click Browse.
' d. Click:
' install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 4. Ensure that the specified drawing document exists.
' 5. Open the Immediate window.
' 6. Substitute your_license_key with your SOLIDWORKS Document
' Manager license key.
' 7. Run the macro.
'
' Postconditions: Examine the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.swdocumentmgr
Imports System.Runtime.InteropServices
Imports System
Imports SolidWorks.Interop
Imports System.Diagnostics
Imports System.Math

```

```
Module Module1
```

```

 Sub Main()
 Dim swClassFact As SwDocumentMgr.SwDMClassFactory
 Dim docMgrApp As SwDocumentMgr.SwDMAApplication
 Dim docDrw As SwDocumentMgr.SwDMDocument13
 Dim res As SwDocumentMgr.SwDmDocumentOpenError

 swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 docMgrApp = swClassFact.GetApplication("your_license_key")
 docDrw = docMgrApp.GetDocument("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", SwDmDocumentType.SwDmDocumentDrawing, True, res)
 Dim sheetNameVar As Object
 Dim sheetCount As Integer

 sheetCount = docDrw.GetSheetCount
 sheetNameVar = docDrw.GetSheetNames

 Dim sheetName As String
 Dim sheetNameCount As Integer

 For sheetNameCount = LBound(sheetNameVar) To UBound(sheetNameVar)

 Dim formatName As String = ""
 Dim status1 As swdocumentmgr.swSheetFormatPathResult

 sheetName = sheetNameVar(sheetNameCount)
 status1 = docDrw.GetSheetFormatPath(sheetName, formatName)

 Debug.Print(sheetName & " ---- " & formatName & vbNewLine)

 Dim status2 As swdocumentmgr.swSheetPropertiesResult
 Dim formatProp As New Object

 status2 = docDrw.GetSheetProperties(sheetName, formatProp)

 Dim para As Double

 para = Round(formatProp(0), 1)

 Dim paperSize As String = ""

 GetPaperSize(para, paperSize)

 Debug.Print("Paper Size = " & paperSize)
 Debug.Print("Size = " & formatProp(1) & " x " & formatProp(2))
 Debug.Print("Scale = " & formatProp(3) & " : " & formatProp(4))

 Dim boolData As Boolean

 boolData = False

 If (formatProp(5) = 1) Then
 boolData = True
 End If

```

```

 Debug.Print("First angle = " & boolData)

 Dim templateSize As String = ""
 para = Round(formatProp(6), 1)
 GetTemplateSize(para, templateSize)
 Debug.Print("Template size = " & templateSize)

 boolData = False

 If (formatProp(7) = 1) Then
 boolData = True
 End If

 Debug.Print("Template visible = " & boolData)
 Debug.Print("=====")

Next

docDrw.CloseDoc()

End Sub

Sub GetPaperSize(ByVal inpNum As Double, ByRef outPaperType As String)
 If (inpNum = 0) Then
 outPaperType = "swDwgPaperASize"
 ElseIf (inpNum = 1) Then
 outPaperType = "swDwgPaperASizeVertical"
 ElseIf (inpNum = 2) Then
 outPaperType = "swDwgPaperBsize"
 ElseIf (inpNum = 3) Then
 outPaperType = "swDwgPaperCsize"
 ElseIf (inpNum = 4) Then
 outPaperType = "swDwgPaperDsize"
 ElseIf (inpNum = 5) Then
 outPaperType = "swDwgPaperEsize"
 ElseIf (inpNum = 6) Then
 outPaperType = "swDwgPaperA4size"
 ElseIf (inpNum = 7) Then
 outPaperType = "swDwgPaperA4sizeVertical"
 ElseIf (inpNum = 8) Then
 outPaperType = "swDwgPaperA3size"
 ElseIf (inpNum = 9) Then
 outPaperType = "swDwgPaperA2size"
 ElseIf (inpNum = 10) Then
 outPaperType = "swDwgPaperA1size"
 ElseIf (inpNum = 11) Then
 outPaperType = "swDwgPaperA0size"
 ElseIf (inpNum = 12) Then
 outPaperType = "swDwgPapersUserDefined"
 End If
End Sub

Sub GetTemplateSize(ByVal inpNum As Double, ByRef outTemplateType As String)
 If (inpNum = 0) Then
 outTemplateType = "swDwgTemplateASize"
 ElseIf (inpNum = 1) Then
 outTemplateType = "swDwgTemplateASizeVertical"
 ElseIf (inpNum = 2) Then

```

```
 outTemplateType = "swDwgTemplateBsize"
 ElseIf (inpNum = 3) Then
 outTemplateType = "swDwgTemplateCsize"
 ElseIf (inpNum = 4) Then
 outTemplateType = "swDwgTemplateDsize"
 ElseIf (inpNum = 5) Then
 outTemplateType = "swDwgTemplateEsize"
 ElseIf (inpNum = 6) Then
 outTemplateType = "swDwgTemplateA4size"
 ElseIf (inpNum = 7) Then
 outTemplateType = "swDwgTemplateA4sizeVertical"
 ElseIf (inpNum = 8) Then
 outTemplateType = "swDwgTemplateA3size"
 ElseIf (inpNum = 9) Then
 outTemplateType = "swDwgTemplateA2size"
 ElseIf (inpNum = 10) Then
 outTemplateType = "swDwgTemplateA1size"
 ElseIf (inpNum = 11) Then
 outTemplateType = "swDwgTemplateA0size"
 ElseIf (inpNum = 12) Then
 outTemplateType = "swDwgTemplateCustom"
 ElseIf (inpNum = 13) Then
 outTemplateType = "swDwgTemplateNone"
 End If
End Sub
End Module
```

## Get eDrawings Data Example (C#)

This example shows how to get eDrawings data from a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

**NOTE:** ISwDMDocument::GetEDrawingsData is obsolete as of SOLIDWORKS Document Manager API 2005 FCS and has not been superseded.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic and
// ensure that the required DLLs have been registered.
// 2. Copy and paste this module into a C# console
// application in Microsoft Visual Studio.
// 3. Ensure that the specified document exists (or point to another document
// that contains eDrawings data).
// 4. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Click Add Reference.
// c. Click Browse.
// d. Click:
// install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 5. Substitute your_license_code with your
// SOLIDWORKS Document Manager license key.
// 6. Compile and run this program in Debug mode.
// Postconditions:
// 1. Inspect the error code in the Output window.
// 2. The eDrawings data is extracted to a temp file in C:\temp.
//-----

using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.swdocumentmgr;
class Program
{
 static void Main(string[] args)
 {
 const string sLicenseKey = "your_license_code";
 //Specify your license key
 const string sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\EDraw\\claw\\claw-
mechanism.sldasm";

 SwDMClassFactory swClassFact = default(SwDMClassFactory);
 SwDMAApplication swDocMgr = default(SwDMAApplication);
 SwDMDocument swDoc = default(SwDMDocument);
 SwDmDocumentType nDocType = 0;
 string sEdrawExt = null;
 SwDmDocumentOpenError nRetVal = default(SwDmDocumentOpenError);
 SwDmEDwgDataError nRetVal2 = default(SwDmEDwgDataError);

 // Determine type of SOLIDWORKS file based on file extension

 if (sDocFileName.EndsWith(".sldprt"))
 {
 nDocType = SwDmDocumentType.swDmDocumentPart;
 sEdrawExt = ".eprt";
 }
 else if (sDocFileName.EndsWith(".sldasm"))
 {
 nDocType = SwDmDocumentType.swDmDocumentAssembly;
 sEdrawExt = ".easm";
 }
 else if (sDocFileName.EndsWith(".slddrw"))
 {
 nDocType = SwDmDocumentType.swDmDocumentDrawing;
 sEdrawExt = ".edrw";
 }
 else
 {
 // Not a SOLIDWORKS file
 nDocType = SwDmDocumentType.swDmDocumentUnknown;
 sEdrawExt = "";

 // So cannot open
 return;
 }

 swClassFact = new SwDMClassFactory();
 swDocMgr = (SwDMAApplication)swClassFact.GetApplication(sLicenseKey);
 swDoc = (SwDMDocument)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
 Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone == nRetVal);

 // Copy eDrawings data to file with eDrawings extension
 nRetVal2 = swDoc.GetEDrawingsData("c:\\temp\\temp." + sEdrawExt);
 }
}
```

```
 Debug.Print("EDrawing error code: " + nRetVal2);
 }
}
```

## Get eDrawings Data Example (VB.NET)

This example shows how to get eDrawings data from a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

**NOTE:** ISwDMDocument::GetEDrawingsData is obsolete as of SOLIDWORKS Document Manager API 2005 FCS and has not been superseded.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
' Getting Started topic and ensure that the
' required DLLs have been registered.
' 2. Copy and paste this module into a VB.NET
' console application in Microsoft Visual Studio.
' 3. Ensure that the specified document exists
' (or point to another document that contains
' eDrawings data).
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll
' reference to the project:
' a. Right-click the solution in Solution
' Explorer.
' b. Click Add Reference.
' c. Click Browse.
' d. Click:
' install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 5. Substitute your_license_code with your
' SOLIDWORKS Document Manager license key.
' 6. Compile and run this program in Debug mode.
'
' Postconditions:
' 1. Inspect the error code in the Output window.
' 2. The eDrawings data is extracted to a temp file in c:\temp.
'-----

Imports SolidWorks.Interop.swdocumentmgr
Module Module1

 Sub main()

 Const sLicenseKey As String = "your_license_code" 'Specify your license key
 Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\EDraw\claw\claw-
mechanism.sldasm"

 Dim swClassFact As SwDMClassFactory
 Dim swDocMgr As SwDMAApplication
 Dim swDoc As SwDMDocument
 Dim nDocType As Integer
 Dim sEdrawExt As String
 Dim nRetVal As SwDmEDwgDataError

 ' Determine type of SOLIDWORKS file based on file extension
 If InStr(LCase(sDocFileName), "sldprt") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentPart
 sEdrawExt = "eprt"
 ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentAssembly
 sEdrawExt = "easm"
 ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentDrawing
 sEdrawExt = "edrw"
 Else

 ' Probably not a SOLIDWORKS file
 nDocType = SwDmDocumentType.swDmDocumentUnknown
 sEdrawExt = ""

 ' so cannot open
 Exit Sub
 End If

 swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 swDocMgr = swClassFact.GetApplication(sLicenseKey)
 swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal) :
Debug.Assert(SwDmDocumentOpenError.swDmDocumentOpenErrorNone = nRetVal)

 ' Copy eDrawings data to file with eDrawings extension
 nRetVal = swDoc.GetEDrawingsData("c:\temp\temp." & sEdrawExt)
 Debug.Print("EDrawing error code: " & nRetVal)

 End Sub
End Module

```

## Get External References Example (C#)

This example shows how to get all of the external references for the base part using the SOLIDWORKS Document Manager API.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started topic and
// ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
// in Microsoft Visual Studio.
// 3. Modify the path of the specified assembly.
// 4. Rename the namespace to match the name of your C# project.
// 5. Add the SolidWorks.Interop.swdocumentmgr.dll
// reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Click Add Reference.
// c. Click Browse.
// d. Click:
// install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 6. Substitute your_license_key with your SOLIDWORKS Document Manager license key.
// 7. Ensure that c:\temp exists.
//
// Postconditions:
// 1. Inspect the Immediate Window for the external references and their
// configurations.
// 2. Inspect c:\temp\extRef.xml for more information about the external
// references.
//-----

using System;
using System.Collections.Generic;
using System.Text;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;

namespace GetExtRef_CSharp
{
 class Program
 {
 static SwDMClassFactory dmClassFact;
 static SwDMAApplication4 dmDocMgr;
 static SwDMDocument18 dmDoc;
 static SwDmDocumentType dmDocType;
 static SwDMSearchOption dmSearchOpt;
 static SwDmDocumentOpenError status;
 static SwDMExternalReferenceOption2 dmExtRefOption;
 static int numExtRefs;

 static SwDmXmlDataError xmlError;

 const string docPath = "public_documents\\samples\\tutorial\\api\\assem2.sldasm";
 const string licenseKey = "your_license_key";

 static void Main()
 {
 setDocType();

 dmClassFact = new SwDMClassFactory();
 dmDocMgr = (SwDMAApplication4)dmClassFact.GetApplication(licenseKey);
 dmDoc = (SwDMDocument18)dmDocMgr.GetDocument(docPath, dmDocType, true, out status);

 if ((dmDoc != null))
 {
 NewMethod();
 dmDoc.GetXmlStream("c:\\temp\\extRef.xml", ref xmlError);
 dmDoc.CloseDoc();
 }
 else
 {
 Debug.Print("Unable to open document. Check 'docPath' variable.");
 }
 }

 public static void NewMethod()
 {
 dmExtRefOption = (SwDMExternalReferenceOption2)dmDocMgr.GetExternalReferenceOptionObject2();
 dmSearchOpt = dmDocMgr.GetSearchOptionObject();

 dmExtRefOption.SearchOption = dmSearchOpt;
 dmExtRefOption.Configuration = "Default";
 dmExtRefOption.NeedSuppress = true;
 numExtRefs = dmDoc.GetExternalFeatureReferences2(dmExtRefOption);

 PrintStrings("FilePath", dmExtRefOption.ExternalReferences);
 }
 }
}
```

```

 PrintStrings("ConfigName", dmExtRefOption.ReferencedConfigurations);

 dmSearchOpt = null;
 dmExtRefOption = null;
 }

 public static void setDocType()
 {
 string typeStr = null;

 typeStr = docPath.Substring((docPath.Length - 6),6);
 typeStr = typeStr.ToUpper();

 if ((typeStr == "SLDPRT"))
 {
 dmDocType = SwDmDocumentType.swDmDocumentPart;
 }
 else if ((typeStr == "SLDASM"))
 {
 dmDocType = SwDmDocumentType.swDmDocumentAssembly;
 }
 }

 public static void PrintStrings(string name, object varInp)
 {
 int I = 0;
 String[] arrInp;
 arrInp = (String[])varInp;
 for (I = 0; I < arrInp.Length; I++)
 {
 string str = null;
 str = arrInp[I];
 Debug.Print(name + " : " + str);
 }
 }
}
}

```

## Get External References Example (VB.NET)

This example shows how to get all of the external references for the base part using the SOLIDWORKS Document Manager API.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
' Getting Started topic and ensure that the
' required DLLs are registered.
' 2. Copy and paste this code into a VB.NET console application
' in Microsoft Visual Studio.
' 3. Modify the path to the specified assembly.
' 4. Add the SolidWorks.Interop.swdocumentmgr.dll
' reference to the project:
' a. Right-click the solution in Solution Explorer.
' b. Click Add Reference.
' c. Click Browse.
' d. Click:
' install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 5. Substitute your_license_key with your SOLIDWORKS Document Manager license key.
' 6. Ensure that c:\temp exists.
' Postconditions:
' 1. Inspect the Immediate Window for the external references and their
' configurations.
' 2. Inspect c:\temp\extRef.xml for more information about the external
' references.
'-----

```

```
Imports SolidWorks.Interop.swdocumentmgr
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Module Module1
```

```

 Dim dmClassFact As SwDMClassFactory
 Dim dmDocMgr As SwDMApplication4
 Dim dmDoc As SwDMDocument18
 Dim dmDocType As SwDMDocumentType
 Dim dmSearchOpt As SwDMSearchOption
 Dim status As SwDMDocumentOpenError
 Dim dmExtRefOption As SwDMExternalReferenceOption2
 Dim numExtRefs As Integer
 Dim xmlError As Integer
 Const docPath As String = "public_documents\samples\tutorial\api\assem2.sldasm"
 Const licenseKey As String = "your_license_key"

```

```
Sub main()
```

```
 setDocType()
```

```

 dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 dmDocMgr = dmClassFact.GetApplication(licenseKey)
 dmDoc = dmDocMgr.GetDocument(docPath, dmDocType, True, status)

```

```
 If Not (dmDoc Is Nothing) Then
```

```

 NewMethod()
 dmDoc.GetXmlStream("c:\temp\extRef.xml", xmlError)
 dmDoc.CloseDoc()
 Else

```

```

 Debug.Print("Unable to open document. Check 'docPath' variable.")
 End If

```

```
End Sub
```

```
Sub NewMethod()
```

```

 dmExtRefOption = dmDocMgr.GetExternalReferenceOptionObject2
 dmSearchOpt = dmDocMgr.GetSearchOptionObject

```

```

 dmExtRefOption.SearchOption = dmSearchOpt
 dmExtRefOption.Configuration = "Default"
 dmExtRefOption.NeedSuppress = True
 numExtRefs = dmDoc.GetExternalFeatureReferences2(dmExtRefOption)

```

```

 Debug.Print("External references:")
 PrintStrings("FilePath", dmExtRefOption.ExternalReferences)
 Debug.Print("")
 Debug.Print("Configurations of external references:")
 PrintStrings("ConfigName", dmExtRefOption.ReferencedConfigurations)

```

```

 dmSearchOpt = Nothing
 dmExtRefOption = Nothing

```

```
End Sub
```

```
Sub setDocType()
```

```
 Dim typeStr As String
```

```
typeStr = Mid$(docPath, (Len(docPath) + 1 - 6), 6)
typeStr = UCase$(typeStr)

If (typeStr = "SLDPRT") Then
 dmDocType = SwDmDocumentType.swDmDocumentPart
ElseIf (typeStr = "SLDASM") Then
 dmDocType = SwDmDocumentType.swDmDocumentAssembly
End If

End Sub

Sub PrintStrings(ByVal name As String, ByVal varInp As Object)

 Dim I As Integer

 For I = LBound(varInp) To UBound(varInp)
 Dim str As String
 str = varInp(I)
 Debug.Print(name & " : " & str)
 Next I

End Sub

End Module
```

## Get External References for Part Example (C#)

This example shows how to get all of the external references for a part document.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Getting Started
// topic and ensure that the required DLLs are registered.
// 2. Copy and paste this code into a C# console application
// in Microsoft Visual Studio.
// 3. Substitute part_path_and_filename with the path and filename
// of a part document containing one or more external references
// to other part documents.
// 4. Rename the namespace to match the name of your C# project.
// 5. Add the SolidWorks.Interop.swdocumentmgr.dll
// reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Click Add Reference.
// c. Click Browse.
// d. Click:
// install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 6. Substitute your_license_key with your SOLIDWORKS Document
// Manager license key.
// 7. Ensure that c:\temp exists.
//
// Postconditions:
// 1. Examine the Immediate Window for the external references.
// 2. Examine c:\temp\extRefPart.xml for more information about the external
// references.
//-----

using System;
using System.Collections.Generic;
using System.Text;
using System.Diagnostics;
using SolidWorks.Interop.swdocumentmgr;

namespace GetExtRef_CSharp
{
 class Program
 {
 static SwDMClassFactory dmClassFact;
 static SwDMAApplication4 dmDocMgr;
 static SwDMDocument18 dmDoc;
 static SwDmDocumentType dmDocType;
 static SwDMSearchOption dmSearchOpt;
 static SwDmDocumentOpenError status;
 static SwDMExternalReferenceOption2 dmExtRefOption;
 static int numExtRefs;
 static SwDmXmlDataError xmlError;
 static string[] extRefs;
 static string[] refConfigs;
 static int[] refIDs;
 static bool featStatus;
 static object featNames;
 static string[] strFeatNames;
 static object featTypes;
 static string[] strFeatTypes;
 static int[] brokenStatus;
 static int numFeat;
 static int i;
 static int j;

 const string docPath = "part_path_and_filename";
 const string licenseKey = "your_license_key";

 static void Main()
 {
 setDocType();

 dmClassFact = new SwDMClassFactory();
 dmDocMgr = (SwDMAApplication4)dmClassFact.GetApplication(licenseKey);
 dmDoc = (SwDMDocument18)dmDocMgr.GetDocument(docPath, dmDocType, true, out status);

 if ((dmDoc != null))
 {
 GetReferences();
 dmDoc.GetXmlStream("c:\\temp\\extRefPart.xml", ref xmlError);
 dmDoc.CloseDoc();
 }
 else
 {
 }
 }
 }
}
```

```

 Debug.Print("Unable to open document. Check 'docPath' variable.");
 }
}

public static void GetReferences()
{
 dmExtRefOption = dmDocMgr.GetExternalReferenceOptionObject2();
 dmExtRefOption.Configuration = "Default";
 dmExtRefOption.NeedSuppress = true;
 dmSearchOpt = dmDocMgr.GetSearchOptionObject();
 dmSearchOpt.SearchFilters = ((int)SwDmSearchFilters.SwDmSearchExternalReference + (int)SwDmSearchFilters.SwDmSearchForPe
 dmExtRefOption.SearchOption = dmSearchOpt;

 // Gets the paths and filenames of the external references, whether the
 // external references are broken, and the names of their
 // referenced configurations
 numExtRefs = dmDocMgr.GetExternalFeatureReferences3(dmExtRefOption);
 extRefs = (string[])dmExtRefOption.ExternalReferences;
 refConfigs = (string[])dmExtRefOption.ReferencedConfigurations;
 brokenStatus = (int[])dmExtRefOption.BrokenStatus;
 refIDs = (int[])dmExtRefOption.IDs;

 if (numExtRefs > 0)
 {
 for (i = 0; i < numExtRefs; i++)
 {
 Debug.Print("External references: " + extRefs[i]);
 Debug.Print(" Broken reference (1=broken; 2=not broken; 0=older version or N/A): " + brokenStatus[i]);
 Debug.Print(" Configuration name: " + refConfigs[i]);
 Debug.Print(" ID: " + refIDs[i]);
 Debug.Print("");
 }

 // Get the referenced feature names and types in the part document
 featStatus = dmExtRefOption.ReferencedFeatures(out featNames, out featTypes);
 strFeatNames = (string[])featNames;
 strFeatTypes = (string[])featTypes;
 Debug.Print("Referenced feature names and types in part document: ");
 numFeat = 0;
 if (strFeatNames.Length > 0)
 {
 for (j = 0; j < extRefs.Length; j++)
 {
 Debug.Print(" Feature name: " + strFeatNames[j]);
 Debug.Print(" Feature type: " + strFeatTypes[j]);
 Debug.Print("");
 numFeat = numFeat + 1;
 }
 }
 }
}

public static void setDocType()
{
 string typeStr = null;

 typeStr = docPath.Substring((docPath.Length - 6),6);
 typeStr = typeStr.ToUpper();

 if ((typeStr == "SLDPRT"))
 {
 dmDocType = SwDmDocumentType.swDmDocumentPart;
 }
 else if ((typeStr == "SLDASM"))
 {
 dmDocType = SwDmDocumentType.swDmDocumentAssembly;
 }
}

public static void PrintStrings(string name, object varInp)
{
 int I = 0;
 String[] arrInp;
 arrInp = (String[])varInp;
 for (I = 0; I < arrInp.Length; I++)
 {
 string str = null;
 str = arrInp[I];
 Debug.Print(name + " : " + str);
 }
}
}
}
}

```

## Get External References for Part Example (VB.NET)

This example shows how to get all of the external references for a part document.

```

Preconditions:
1. Read the SOLIDWORKS Document Manager API Getting Started topic
 and ensure that the required DLLs are registered.
2. Copy and paste this code into a VB.NET console application
 in Microsoft Visual Studio.
3. Substitute part_path_and_filename with the filename
 of a part document containing one or more external references to other
 part documents.
4. Add the SolidWorks.Interop.swdocumentmgr.dll
 reference to the project:
 a. Right-click the solution in Solution Explorer.
 b. Click Add Reference.
 c. Click Browse.
 d. Click:
 install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
5. Substitute your_license_key with your SOLIDWORKS Document
 Manager license key.
6. Ensure that c:\temp exists.

Postconditions:
1. Examine the Immediate Window for information about the external references.
2. Examine c:\temp\extRefPart.xml for more information about the
 external references.

```

```
Imports SolidWorks.Interop.swdocumentmgr
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Module Module1
```

```

Dim dmClassFact As SwDMClassFactory
Dim dmDocMgr As SwDMApplication4
Dim dmDoc As SwDMDocument18
Dim dmDocType As SwDmDocumentType
Dim dmSearchOpt As SwDMSearchOption
Dim status As SwDmDocumentOpenError
Dim dmExtRefOption As SwDMExternalReferenceOption2
Dim numExtRefs As Integer
Dim xmlError As Integer
Dim extRefs() As String
Dim refConfigs() As String
Dim refIDs() As Integer
Dim featStatus As Boolean
Dim featNames() As String
Dim featTypes() As String
Dim brokenStatus() As Integer
Dim numFeat As Integer
Dim i As Integer
Dim j As Integer
Const docPath As String = "part_path_and_filename "
Const licenseKey As String = "your_license_key"

```

```
Sub Main()
```

```
 setDocType()
```

```

 dmClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 dmDocMgr = dmClassFact.GetApplication(licenseKey)
 dmDoc = dmDocMgr.GetDocument(docPath, dmDocType, True, status)

```

```
 If Not (dmDoc Is Nothing) Then
```

```

 GetReferences()
 dmDoc.GetXmlStream("c:\temp\extRefPart.xml", xmlError)
 dmDoc.CloseDoc()
 Else

```

```
 Debug.Print("Unable to open document. Check 'docPath' variable.")
```

```
 End If
```

```
End Sub
```

```
Sub GetReferences()
```

```

 ' Get external references
 dmExtRefOption = dmDocMgr.GetExternalReferenceOptionObject2
 dmExtRefOption.Configuration = "Default"
 dmExtRefOption.NeedSuppress = True
 dmSearchOpt = dmDocMgr.GetSearchOptionObject()
 dmSearchOpt.SearchFilters = (SwDmSearchFilters.SwDmSearchExternalReference + SwDmSearchFilters.SwDmSearchForPart)
 dmExtRefOption.SearchOption = dmSearchOpt

 ' Gets the paths and filenames of the external references,
 ' whether the external references are broken, and the names of their
 ' referenced configurations

```

```

numExtRefs = dmDoc.GetExternalFeatureReferences3(dmExtRefOption)
extRefs = dmExtRefOption.ExternalReferences
refConfigs = dmExtRefOption.ReferencedConfigurations
brokenStatus = dmExtRefOption.BrokenStatus
refIDs = dmExtRefOption.IDs

If (numExtRefs > 0) Then

 For i = 0 To numExtRefs - 1
 Debug.Print("External reference: " & extRefs(i))
 Debug.Print(" Broken reference (1=broken, 2=not broken, 0 = older version or N/A): " & brokenStatus(i))
 Debug.Print(" Configuration name: " & refConfigs(i))
 Debug.Print(" ID: " & refIDs(i))
 Debug.Print("")
 Next

 ' Get the referenced feature names and types in the part document
 featStatus = dmExtRefOption.ReferencedFeatures(featNames, featTypes)
 Debug.Print("Referenced feature names and types in part document: ")
 numFeat = 0
 If (featStatus And (featNames IsNot Nothing)) Then
 For j = LBound(extRefs) To UBound(extRefs)
 Debug.Print(" Feature name: " & featNames(numFeat))
 Debug.Print(" Feature type: " & featTypes(numFeat))
 Debug.Print("")
 numFeat = numFeat + 1
 Next j
 Else
 Debug.Print(" No referenced features in model document.")
 End If
Else
 Debug.Print("No external references in the part document.")
End If

End Sub

Sub setDocType()

 Dim typeStr As String

 typeStr = Mid$(docPath, (Len(docPath) + 1 - 6), 6)
 typeStr = UCase$(typeStr)

 If (typeStr = "SLDPRT") Then
 dmDocType = SwDmDocumentType.swDmDocumentPart
 ElseIf (typeStr = "SLDASM") Then
 dmDocType = SwDmDocumentType.swDmDocumentAssembly
 End If

End Sub

End Module

```

## Get Hyperlinks Example (C#)

This example shows how to get the hyperlinks for a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API
// Getting Started topic and ensure that
// the required DLLs have been registered.
// 2. Copy and paste this module into a C#
// console application in Microsoft Visual Studio.
// 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
// a. Right-click the solution in Solution Explorer.
// b. Click Add Reference.
// c. Click Browse.
// d. Click:
// install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
// 4. Substitute your_license_code with your
// SOLIDWORKS Document Manager license key.
// 5. Substitute model_with_hyperlinks with the path to a model that
// contains one or more hyperlinks.
// 6. Compile and run this program in Debug mode.
//
// Postconditions: Inspect the Output window and the
// code to see how the API is used.
//-----
```

```
using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.swdocumentmgr;

class Program
{

 public static void ProcessDocHyperlinks(SwDMDocument swDoc)
 {
 int i;
 int count = swDoc.GetHyperLinksCount();
 Debug.Print(" Hyperlinks: " + count);

 for (i = 0; i < count; i++)
 {
 Debug.Print(" " + swDoc.GetHyperLinkAt(i));
 }

 Debug.Print("");
 }

 static void Main(string[] args)
 {

 const string slicenseKey = "your_license_code";
 const string sDocFileName = "model_with_hyperlinks";

 SwDMClassFactory swClassFact = default(SwDMClassFactory);
 SwDMAApplication swDocMgr = default(SwDMAApplication);
 SwDMDocument swDoc = default(SwDMDocument);
 SwDMConfigurationMgr swCfgMgr = default(SwDMConfigurationMgr);
 SwDmDocumentType nDocType = 0;
 SwDmDocumentOpenError nRetVal = 0;

 // Determine type of SOLIDWORKS file based on file extension
 if (sDocFileName.EndsWith("sldprt"))
 {
 nDocType = SwDmDocumentType.swDmDocumentPart;
 }
 else if (sDocFileName.EndsWith("sldasm"))
 {
 nDocType = SwDmDocumentType.swDmDocumentAssembly;
 }
 else if (sDocFileName.EndsWith("slddrw"))
 {
 nDocType = SwDmDocumentType.swDmDocumentDrawing;
 }
 else
 {
 // Not a SOLIDWORKS file
 nDocType = SwDmDocumentType.swDmDocumentUnknown;

 // So cannot open
 return;
 }
 }
}
```

```
swClassFact = new SwDMClassFactory();
swDocMgr = swClassFact.GetApplication(sLicenseKey);
swDoc = (SwDMDocument)swDocMgr.GetDocument(sDocFileName, nDocType, false, out nRetVal);
swCfgMgr = swDoc.ConfigurationManager;

Debug.Print("File = " + swDoc.FullName);

ProcessDocHyperlinks(swDoc);
}
```

## Get Hyperlinks Example (VB.NET)

This example shows how to get the hyperlinks for a SOLIDWORKS document using the SOLIDWORKS Document Manager API.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
' Getting Started topic and ensure that
' the required DLLs have been registered.
' 2. Copy and paste this module into a VB.NET
' console application in Microsoft Visual Studio.
' 3. Add the SolidWorks.Interop.swdocumentmgr.dll reference to the project:
' a. Right-click the solution in Solution Explorer.
' b. Click Add Reference.
' c. Click Browse.
' d. Click:
' install_dir\api\redist\SolidWorks.Interop.swdocumentmgr.dll
' 4. Substitute your_license_code with your
' SOLIDWORKS Document Manager license key.
' 5. Substitute model_with_hyperlinks with the path to a model that
' contains one or more hyperlinks.
' 6. Compile and run this program in Debug mode.
'
' Postconditions: Inspect the Output window and the
' code to see how the API is used.
'-----

```

```
Imports SolidWorks.Interop.swdocumentmgr
Module Module1
```

```
 Sub ProcessDocHyperlinks(ByVal swDoc As SwDMDocument)
```

```
 Dim i As Integer
 Dim count As Integer = swDoc.GetHyperLinksCount

 If count > 0 Then
 Debug.Print(" Hyperlinks: " & count)
 For i = 0 To count - 1
 Debug.Print(" " & swDoc.GetHyperLinkAt(i))
 Next
 Else
 Debug.Print("No hyperlinks detected.")
 End If

 Debug.Print("")
 End Sub
```

```
 Sub main()
```

```
 Const sLicenseKey As String = "your_license_code" 'Specify your license key
 Const sDocFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\EDraw\claw\claw-
mechanism.sldasm"
```

```
 Dim swClassFact As SwDMClassFactory
 Dim swDocMgr As SwDMAApplication
 Dim swDoc As SwDMDocument
 Dim swCfgMgr As SwDMConfigurationMgr
 Dim nDocType As Integer
 Dim nRetVal As Integer

 ' Determine type of SOLIDWORKS file based on file extension
 If InStr(LCase(sDocFileName), "sldprt") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentPart
 ElseIf InStr(LCase(sDocFileName), "sldasm") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentAssembly
 ElseIf InStr(LCase(sDocFileName), "slddrw") > 0 Then
 nDocType = SwDmDocumentType.swDmDocumentDrawing
 Else

 ' Probably not a SOLIDWORKS file
 nDocType = SwDmDocumentType.swDmDocumentUnknown

 ' so cannot open
 Exit Sub
 End If
```

```
 swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 swDocMgr = swClassFact.GetApplication(sLicenseKey)
 swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, False, nRetVal)
 swCfgMgr = swDoc.ConfigurationManager
```

```
 Debug.Print("File = " & swDoc.FullName)
```

```
 ProcessDocHyperlinks(swDoc)
```

End Sub

End Module

## Get Linked Display States Example (C#)

This example gets an assembly and reports on its active configuration and display states.

```
// -----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API
// Getting Started topic in the API help
// and ensure that the required DLLs have been registered.
// 2. Open SOLIDWORKS and copy the code below to a C# macro.
// 3. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
// interop assembly is loaded in your project. (Right-click your
// project in Project Explorer, click Add Reference,
// click the interop assembly in the .NET tab, or browse
// for the DLL in install_dir\api\redist).
// 4. Substitute your license key for your_license_key in the code.
// 5. Open the Immediate window.
//
// Postconditions: Examine the Immediate Window for output.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

partial class SolidWorksMacro
{
 public SldWorks swApp;
 public void Main()
 {
 SwDMClassFactory swCf = default(SwDMClassFactory);
 swCf = new SwDMClassFactory();
 SwDMAApplication swDocMgr = default(SwDMAApplication);
 swDocMgr = swCf.GetApplication("your_license_key");
 int i = 0;
 int j = 0;
 SwDMDocument14 swDoc12 = default(SwDMDocument14);
 SwDmDocumentOpenError res = default(SwDmDocumentOpenError);
 SwDmDocumentType dt = default(SwDmDocumentType);
 dt = SwDmDocumentType.swDmDocumentAssembly;
 string filename;

 Debug.Print("Opening an assembly...");
 filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\motor casing.sldasm";
 swDoc12 = (SwDMDocument14)swDocMgr.GetDocument(filename, dt, false, out res);
 if (swDoc12 == null | (res != SwDmDocumentOpenError.swDmDocumentOpenErrorNone))
 {
 Debug.Print("Error opening file...");
 return;
 }
 SwDMConfiguration12 activeConfig = default(SwDMConfiguration12);
 string activeConfigName = null;
 SwDMConfigurationMgr configMgr = default(SwDMConfigurationMgr);
 configMgr = swDoc12.ConfigurationManager;
 activeConfigName = configMgr.GetActiveConfigurationName();
 Debug.Print("Getting the active configuration: " + activeConfigName);
 activeConfig = (SwDMConfiguration12)configMgr.GetConfigurationByName(activeConfigName);
 if (activeConfig == null)
 {
 Debug.Print("Error getting the active configuration...");
 return;
 }
 Debug.Print("ID of " + activeConfig.Name2 + ": " + activeConfig.GetID());
 }
}
```

```

Debug.Print("");

int numLinkedDisplayStates = 0;
String[] ldsVariant = null;
string lds = null;
Boolean[] compVisibleList = null;
string[] compFileDirectory = null;
Object ldsObject = null;

numLinkedDisplayStates = activeConfig.GetLinkedDisplayStates(out ldsObject);
Debug.Print("Number of linked display states of " + activeConfigName + ": " + numLinkedDisplayStates);

// Getting all the components for one linked display state
ldsVariant = (String[])ldsObject;
Object vis = null;
Object fd = null;
lds = ldsVariant[0];
activeConfig.GetComponentVisibleByDisplayStateName(lds, out vis, out fd);
Debug.Print("");
Debug.Print("Getting the paths and file names of all of the components in the linked display state, " + lds + ": ");
Debug.Print("");
compVisibleList = (Boolean[])vis;
compFileDirectory = (String[])fd;
for (j = 0; j <= compFileDirectory.GetUpperBound(0); j++)
{
 Debug.Print("Component: " + compFileDirectory[j]);
 Debug.Print(" Is visible? " + compVisibleList[j]);
}
//Getting all the components in the configuration
Debug.Print("");
Debug.Print("Getting the names of all of the components in the FeatureManager design tree in C:\\Program
Files\\SOLIDWORKS Corp\\SOLIDWORKS\\api\\redist\\ " + activeConfig.Name2 + ": ");
Debug.Print("");
Object[] vComponents = null;
vComponents = (Object[])activeConfig.GetComponents();
if (!(vComponents == null))
{
 SwDMComponent11 swDmComponent = default(SwDMComponent11);
 for (i = 0; i <= vComponents.GetUpperBound(0); i++)
 {
 swDmComponent = (SwDMComponent11)vComponents[i];
 Debug.Print(" Component name: " + swDmComponent.Name3);
 Debug.Print(" For a selective open with OpenDoc7, use: " + swDmComponent.SelectName2);
 }
}
swDoc12.CloseDoc();
}
}

```

## Get Linked Display States Example (VB.NET)

This example gets an assembly and reports on its active configuration and display states.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
' Getting Started topic and ensure that the
' required DLLs are registered.
' 2. Open SOLIDWORKS and copy the code below to a VB.NET macro.
' 3. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
' interop assembly is loaded in the project.
' (right-click the project in Project Explorer, click
' Add Reference, click the interop assembly in the
' .NET tab, or browse for the DLL in
' install_dir\api\redist).
' 4. Substitute your license key for your_license_key in
' the code.
' 5. Open the Immediate window.
'-----
' Postconditions: Examine the Immediate Window for output.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

 Sub main()
 Dim swCf As SwDMClassFactory
 swCf = New SwDMClassFactory
 Dim swDocMgr As SwDMAApplication

 swDocMgr = swCf.GetApplication("your_license_key")

 Dim i As Integer, j As Integer
 Dim swDoc12 As SwDMDocument12
 Dim res As SwDmDocumentOpenError
 Dim dt As SwDmDocumentType
 dt = SwDmDocumentType.swDmDocumentAssembly
 Dim filename As String
 Debug.Print("Opening an assembly...")
 filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\motor casing.sldasm"
 swDoc12 = swDocMgr.GetDocument(filename, dt, False, res)
 If swDoc12 Is Nothing Or (res <> SwDmDocumentOpenError.swDmDocumentOpenErrorNone) Then
 Debug.Print("Error opening file....")
 Exit Sub
 End If
 Dim activeConfig As SwDMConfiguration12
 Dim activeConfigName As String
 Dim configMgr As SwDMConfigurationMgr
 configMgr = swDoc12.ConfigurationManager
 activeConfigName = configMgr.GetActiveConfigurationName
 Debug.Print("Getting the active configuration: " & activeConfigName)
 activeConfig = configMgr.GetConfigurationByName(activeConfigName)
 If activeConfig Is Nothing Then
 Debug.Print("Error getting the active configuration...")
 Exit Sub
 End If
 Debug.Print("ID of " & activeConfig.Name2 & ": " & activeConfig.GetID)
 Debug.Print("")
 Dim numLinkedDisplayStates As Integer
 Dim ldsVariant As Object
 Dim lds As String
 End Sub
End Class

```

```

Dim compVisibleList As Object
Dim compFileDirectory As Object
ldsVariant = Nothing
numLinkedDisplayStates = activeConfig.GetLinkedDisplayStates(ldsVariant)
Debug.Print("Number of linked display states of " & activeConfigName & ": " & numLinkedDisplayStates)
' Getting all the components for one linked display state
compVisibleList = Nothing
compFileDirectory = Nothing
lds = ldsVariant(0)
activeConfig.GetComponentVisibleByDisplayStateName(lds, compVisibleList, compFileDirectory)
Debug.Print("")
Debug.Print("Getting the paths and file names of all of the components in the linked display state, " & lds & ": ")
Debug.Print("")
For j = 0 To UBound(compFileDirectory)
 Debug.Print("Component: " & compFileDirectory(j))
 Debug.Print(" Is visible? " & compVisibleList(j))
Next 'component
'Getting all the components in the configuration
Debug.Print("")
Debug.Print("Getting the names of all of the components in the FeatureManager design tree for " & activeConfig.Name2 & ": ")
")
Debug.Print("")
Dim vComponents As Object
vComponents = activeConfig.GetComponents
If Not (IsNothing(vComponents)) Then
 Dim swDmComponent As SwDMComponent11
 For i = 0 To UBound(vComponents)
 swDmComponent = vComponents(i)
 Debug.Print(" Component name: " & swDmComponent.Name3)
 Debug.Print(" For a selective open with OpenDoc7, use: " & swDmComponent.SelectName2)
 Next 'component
End If
swDoc12.CloseDoc()
End Sub

Public swApp As SldWorks
End Class

```

## Get Sheet IDs Example (C#)

This example shows how to get the IDs of drawing sheets.

```
//-----
// Preconditions:
// 1. Read the SOLIDWORKS Document Manager API Help
// Getting Started topic and ensure that the required DLLs
// have been registered.
// 2. Open SOLIDWORKS and copy the code below to a C# macro.
// 3. Convert the drawing document specified in sDocFileName
// to the latest supported version by opening and saving it
// to another name in SOLIDWORKS.
//
// NOTE: Because this document is used elsewhere, do not
// save any changes when closing it.
//
// 4. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
// interop assembly is loaded in the project.
// (Right-click the project in Project Explorer,
// click Add Reference, click the interop assembly in
// the .NET tab, or browse for the DLL in
// install_dir\api\redist directory).
// 5. Substitute your license key for your_license_key
// in the code.
//
// Postconditions: Inspect the Immediate Window for the sheet IDs.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swdocumentmgr;
using System;
using System.Diagnostics;

partial class SolidWorksMacro
{
 public void Main()
 {
 SwDMClassFactory swClassFact = default(SwDMClassFactory);
 SwDMAApplication swDocMgr = default(SwDMAApplication);
 SwDMDocument14 swDoc = default(SwDMDocument14);
 string sDocFileName = null;
 SwDmDocumentType nDocType = default(SwDmDocumentType);
 SwDmDocumentOpenError nRetVal = default(SwDmDocumentOpenError);
 string sLicenseKey = null;

 // Specify your license key
 sLicenseKey = "your_license_key";

 // If this drawing document doesn't exist on your system,
 // then substitute the name of drawing document that does
 sDocFileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";

 nDocType = SwDmDocumentType.swDmDocumentDrawing;
 swClassFact = new SwDMClassFactory();
 swDocMgr = swClassFact.GetApplication(sLicenseKey);
 swDoc = (SwDMDocument14)swDocMgr.GetDocument(sDocFileName, nDocType, true, out nRetVal);
 if ((swDoc == null))
 {
 System.Windows.Forms.MessageBox.Show("Unable to open document. Correct path to file or register Document Manager
DLL.");
 }

 SwDMSheet3 Sheet = default(SwDMSheet3);
 object[] Sheets = null;
 int i = 0;
 Sheets = (Object[])swDoc.GetSheets();
 for (i = 0; i <= Sheets.GetUpperBound(0); i++)
 }
}
```

```
 {
 Sheet = (SwDMSheet3)Sheets[i];
 Debug.Print(Sheet.Name + " ID: " + Sheet.GetID());
 }
 swDoc.CloseDoc();
}
public SldWorks swApp;
}
```

## Get Sheet IDs Example (VB.NET)

This example shows how to get the IDs of drawing sheets.

```

'-----
' Preconditions:
' 1. Read the SOLIDWORKS Document Manager API
' Help Getting Started topic and ensure that
' the required DLLs are registered.
' 2. Open SOLIDWORKS and copy the code below to a VB.NET macro.
' 3. Convert the drawing document specified
' in sDocFileName to the latest supported
' version by opening and saving it to another name
' in SOLIDWORKS.
'
' NOTE: Because this document is used elsewhere, do not
' save any changes when closing it.
'
' 4. Ensure that the latest SolidWorks.Interop.swdocumentmgr.dll
' interop assembly is loaded in the project. (Right-click
' the project in Project Explorer, click Add Reference,
' click the interop assembly in the .NET tab, or browse
' for the DLL in install_dir\api\redist.)
' 5. Substitute your license key for your_license_key in the code.
'
' Postconditions: Inspect the Immediate Window for the sheet IDs.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdocumentmgr
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
 Sub main()
 Dim swClassFact As SwDMClassFactory
 Dim swDocMgr As SwDMAApplication
 Dim swDoc As SwDMDocument14
 Dim sDocFileName As String
 Dim nDocType As SwDmDocumentType
 Dim nRetVal As SwDmDocumentOpenError
 Dim sLicenseKey As String

 ' Specify your license key
 sLicenseKey = "your_license_key"

 ' If this drawing document doesn't exist on your system,
 ' then substitute the name of drawing document that does
 sDocFileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\foodprocessor.slddrw"
 nDocType = SwDmDocumentType.swDmDocumentDrawing
 swClassFact = CreateObject("SwDocumentMgr.SwDMClassFactory")
 swDocMgr = swClassFact.GetApplication(sLicenseKey)
 swDoc = swDocMgr.GetDocument(sDocFileName, nDocType, True, nRetVal)
 If (swDoc Is Nothing) Then
 MsgBox("Unable to open document. Correct path to file or register Document Manager DLL.")
 End If
 Dim Sheet As SwDMSheet3
 Dim Sheets As Object
 Dim i As Integer
 Sheets = swDoc.GetSheets
 For i = 0 To UBound(Sheets)
 Sheet = Sheets(i)
 Debug.Print(Sheet.Name & " ID: " & Sheet.GetID)
 Next
 swDoc.CloseDoc()
 End Sub

 Public swApp As SldWorks
End Class

```